

CS1006: Data Structures — CIE 2 Master Notes

Prepared by Francis Reuben R.

RV University, School of CSE | Bengaluru

Table of Contents

- Module 2: Linked Lists and Polynomials
 - 1.1 Introduction to Linked Lists
 - 1.2 Types of Linked Lists
 - 1.3 Singly Linked List — All Operations
 - 1.4 Doubly Linked List — All Operations
 - 1.5 Circular Singly Linked List — All Operations (TIER 1)
 - 1.6 Circular Doubly Linked List — All Operations (TIER 1)
 - 1.7 Polynomial Representation using Singly Linked List
 - 1.8 Polynomial Operations using Circular Linked List (TIER 1)
 - 1.9 Polynomial Operations using Queue (TIER 1)
- Module 3: Stacks and Queues
 - 2.1 Stack — Array and Linked List Implementation
 - 2.2 Infix to Postfix Conversion (TIER 1)
 - 2.3 Postfix Evaluation (TIER 1)

- 2.4 Queue — Array and Linked List Implementation
 - 2.5 Circular Queue (TIER 1)
 - 2.6 Priority Queue
 - Module 4: Sorting, Searching, and Hashing
 - 3.1 Linear Search
 - 3.2 Binary Search (TIER 1)
 - 3.3 Bubble Sort (TIER 1)
 - 3.4 Selection Sort (TIER 1)
 - 3.5 Insertion Sort (TIER 1)
 - 3.6 Hashing (TIER 1)
-

MODULE 2: LINKED LISTS AND POLYNOMIALS

1.1 Introduction to Linked Lists

Priority: Tier 3

Why Linked Lists Exist

An array stores elements in contiguous memory locations, which provides $O(1)$ random access but introduces two critical limitations:

- **Fixed size at compile time.** You must declare the maximum number of elements upfront. This wastes memory if the actual count is far less, and crashes the program if the count exceeds the declared maximum.
- **Expensive insertion and deletion.** Inserting an element at position k in an n -element array requires shifting all elements from k onward by one position, costing $O(n)$ time per operation. Deleting similarly requires shifting.

A linked list solves both problems by storing each element in an independently allocated node in the heap. Each node contains the data and a pointer to the next node. The nodes are connected via pointers, not by physical memory adjacency.

Key properties of linked lists: - Dynamic size: nodes are allocated and freed at runtime using malloc and free - $O(1)$ insertion and deletion once the target position is known - No wasted pre-allocated space - Sequential access only (no random access by index) - Extra memory for pointer field(s) in each node

Memory Layout Comparison

ARRAY (contiguous):

Index: [0] [1] [2] [3] [4]

Data: [10] [20] [30] [40] [50]

Addr: 100 104 108 112 116 <- each int is 4 bytes apart

LINKED LIST (scattered):

Node at addr 200: data=10, next=450

Node at addr 450: data=20, next=130

Node at addr 130: data=30, next=780

Node at addr 780: data=40, next=320

Node at addr 320: data=50, next=NULL

1.2 Types of Linked Lists

Priority: Tier 3

There are four fundamental types, each adding a structural feature:

Type	Direction	Last Node Points To	Extra Pointer?
Singly Linked List	Forward only	NULL	No
Doubly Linked List	Forward and backward	NULL	Yes (prev)
Circular Singly Linked List	Forward, wraps around	Head	No
Circular Doubly Linked List	Both directions, wraps	Head (next) and Tail (prev)	Yes (prev)

Visual Overview

SINGLY:

head -> [10|->] -> [20|->] -> [30|->] -> [40|NULL]

DOUBLY:

head -> [NULL|10|->] <-> [<-|20|->] <-> [<-|30|->] <-> [<-|40|NULL]

CIRCULAR SINGLY:

head -> [10|->] -> [20|->] -> [30|->] -> [40|->] --+



CIRCULAR DOUBLY:

head -> [<-|10|->] <-> [<-|20|->] <-> [<-|30|->] <-> [<-|40|->]



1.3 Singly Linked List

Priority: Tier 2

Node Structure

A singly linked list node holds data (any type, here int) and a next pointer to the following node.

```
struct Node {  
    int data;  
    struct Node *next;  
};  
typedef struct Node* NodePtr;
```

Node layout:

```
+-----+-----+
| data | next |
+-----+-----+
int   pointer
```

Create (Insert First Node)

Algorithm: 1. Allocate a new node using malloc 2. Set newNode->data to the value 3. Set newNode->next to NULL 4. Set head to newNode

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node* createNode(int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = NULL;
    return newNode;
}
```

Insert at Beginning

Algorithm: 1. Create a new node with the given value 2. Set newNode->next to current head 3. Set head to newNode

```

struct Node* insertBeginning(struct Node* head, int val) {
    struct Node* newNode = createNode(val); // Line 1
    newNode->next = head;                    // Line 2
    head = newNode;                          // Line 3
    return head;                             // Line 4
}

```

Line-by-line explanation: 1. Allocate a new node and store the value in it 2. Point new node's next to the current first node (old head) 3. Update head to point to the new node 4. Return the updated head pointer to the caller

Before and After:

Before: head -> [20] -> [30] -> NULL

Insert 10 at beginning

After: head -> [10] -> [20] -> [30] -> NULL

Dry Run (Insert 10, 20, 30 at beginning): - Step 1: Insert 10. head = NULL. newNode(10)->next = NULL. head = node(10) - List: head -> [10|NULL] - Step 2: Insert 20. newNode(20)->next = node(10). head = node(20) - List: head -> [20|->] -> [10|NULL] - Step 3: Insert 30. newNode(30)->next = node(20). head = node(30) - List: head -> [30|->] -> [20|->] -> [10|NULL]

Insert at End

Algorithm: 1. Create a new node 2. If head is NULL, set head to newNode and return 3. Traverse to the last node (node whose next is NULL) 4. Set lastNode->next to newNode

```

struct Node* insertEnd(struct Node* head, int val) {
    struct Node* newNode = createNode(val); // Line 1

```

```

if (head == NULL) {           // Line 2
    return newNode;           // Line 3
}
struct Node* temp = head;      // Line 4
while (temp->next != NULL) {    // Line 5
    temp = temp->next;          // Line 6
}
temp->next = newNode;          // Line 7
return head;                   // Line 8
}

```

Line-by-line explanation: 1. Allocate new node with val 2. Check if list is empty 3. If empty, the new node is the only node; return it as head 4. Start traversal from head 5. Loop until we reach the last node (next is NULL) 6. Advance temp to the next node 7. Connect last node to new node 8. Return unchanged head

Insert at Position

Algorithm: 1. If position is 1, use insertBeginning 2. Traverse to node at position-1 (the node just before target) 3. Set newNode->next = prev->next 4. Set prev->next = newNode

```

struct Node* insertAtPosition(struct Node* head, int val, int pos) {
    if (pos == 1) return insertBeginning(head, val); // Line 1
    struct Node* newNode = createNode(val);          // Line 2
    struct Node* temp = head;                         // Line 3
    for (int i = 1; i < pos - 1 && temp != NULL; i++) { // Line 4
        temp = temp->next;                             // Line 5
    }
    if (temp == NULL) { printf("Invalid position\n"); return head; } // Line 6
}

```

```
newNode->next = temp->next;          // Line 7
temp->next = newNode;                // Line 8
return head;                        // Line 9
}
```

Diagram — Insert 25 at position 3:

Before: head -> [10] -> [20] -> [30] -> [40] -> NULL

^

pos=3 means insert between [20] and [30]

Step 1: temp moves to node at pos-1=2, which is node(20)

Step 2: newNode(25)->next = temp->next = node(30)

Step 3: temp->next = newNode(25)

After: head -> [10] -> [20] -> [25] -> [30] -> [40] -> NULL

Traverse (Display)

```
void traverse(struct Node* head) {
    struct Node* temp = head;    // Line 1
    while (temp != NULL) {       // Line 2
        printf("%d -> ", temp->data); // Line 3
        temp = temp->next;        // Line 4
    }
    printf("NULL\n");            // Line 5
}
```

Delete by Value

Algorithm: 1. If head's data matches, move head forward and free old head 2. Otherwise, traverse to find the node just before the target 3. Link that node's next to skip the target 4. Free the target node

```
struct Node* deleteByValue(struct Node* head, int val) {  
    if (head == NULL) return NULL;           // Line 1  
    if (head->data == val) {                   // Line 2  
        struct Node* temp = head;            // Line 3  
        head = head->next;                    // Line 4  
        free(temp);                           // Line 5  
        return head;                          // Line 6  
    }  
    struct Node* prev = head;                 // Line 7  
    struct Node* curr = head->next;           // Line 8  
    while (curr != NULL && curr->data != val) { // Line 9  
        prev = curr;                         // Line 10  
        curr = curr->next;                   // Line 11  
    }  
    if (curr == NULL) { printf("Not found\n"); return head; } // Line 12  
    prev->next = curr->next;                  // Line 13  
    free(curr);                              // Line 14  
    return head;                             // Line 15  
}
```

Search

```
int search(struct Node* head, int val) {  
    struct Node* temp = head;
```

```
int pos = 1;
while (temp != NULL) {
    if (temp->data == val) return pos;
    temp = temp->next;
    pos++;
}
return -1; // not found
}
```

Time Complexity Summary – Singly Linked List

Operation	Time Complexity	Reason
Insert at Beginning	$O(1)$	No traversal needed
Insert at End	$O(n)$	Must traverse to last node
Insert at Position k	$O(k)$	Traverse to position k-1
Delete by Value	$O(n)$	May need to scan all nodes
Search	$O(n)$	Sequential scan
Traverse	$O(n)$	Visit every node
Space (all)	$O(n)$	One node per element

1.4 Doubly Linked List

Priority: Tier 2

Node Structure

A doubly linked list node has three fields: a prev pointer to the previous node, data, and a next pointer to the next node.

```
struct DNode {  
    int data;  
    struct DNode *prev;  
    struct DNode *next;  
};
```

Node layout:

```
+-----+-----+-----+  
| prev | data | next |  
+-----+-----+-----+  
pointer  int  pointer
```

Full doubly list (head to tail):

NULL <- [10] <-> [20] <-> [30] <-> [40] -> NULL

head = node(10)

Insert at Beginning

```
struct DNode* dInsertBeginning(struct DNode* head, int val) {  
    struct DNode* newNode = (struct DNode*)malloc(sizeof(struct DNode)); // Line 1  
    newNode->data = val;           // Line 2  
    newNode->prev = NULL;         // Line 3  
    newNode->next = head;         // Line 4
```

```

if (head != NULL)      // Line 5
    head->prev = newNode; // Line 6
return newNode;        // Line 7
}

```

Line-by-line: 1. Allocate new node 2. Store the value 3. New node has no predecessor – prev is NULL 4. New node's next points to old head 5. If list was not empty, update old head's prev 6. Old head's prev now points back to new node 7. Return new node as the updated head

Diagram:

Before: head -> [NULL|20|->] <-> [<-|30|NULL]

Insert 10:

After: head -> [NULL|10|->] <-> [<-|20|->] <-> [<-|30|NULL]

Insert at End

```

struct DNode* dInsertEnd(struct DNode* head, int val) {
    struct DNode* newNode = (struct DNode*)malloc(sizeof(struct DNode));
    newNode->data = val;
    newNode->next = NULL;
    if (head == NULL) {
        newNode->prev = NULL;
        return newNode;
    }
    struct DNode* temp = head;
    while (temp->next != NULL) temp = temp->next; // traverse to last
    temp->next = newNode;
    newNode->prev = temp;
}

```

```
    return head;
}
```

Insert at Position

```
struct DNode* dInsertAtPos(struct DNode* head, int val, int pos) {
    if (pos == 1) return dInsertBeginning(head, val);
    struct DNode* newNode = (struct DNode*)malloc(sizeof(struct DNode));
    newNode->data = val;
    struct DNode* temp = head;
    for (int i = 1; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;
    if (temp == NULL) return head; // invalid position
    newNode->next = temp->next;
    newNode->prev = temp;
    if (temp->next != NULL)
        temp->next->prev = newNode;
    temp->next = newNode;
    return head;
}
```

Delete by Value

```
struct DNode* dDeleteByValue(struct DNode* head, int val) {
    struct DNode* curr = head;
    while (curr != NULL && curr->data != val)
        curr = curr->next;
    if (curr == NULL) { printf("Not found\n"); return head; }
    if (curr->prev != NULL)
        curr->prev->next = curr->next;
```

```

else
    head = curr->next; // deleting head node
if (curr->next != NULL)
    curr->next->prev = curr->prev;
free(curr);
return head;
}

```

Traverse Forward and Backward

```

void dTraverseForward(struct DNode* head) {
    struct DNode* temp = head;
    while (temp != NULL) { printf("%d <-> ", temp->data); temp = temp->next; }
    printf("NULL\n");
}

```

```

void dTraverseBackward(struct DNode* head) {
    struct DNode* temp = head;
    while (temp->next != NULL) temp = temp->next; // go to last node
    while (temp != NULL) { printf("%d <-> ", temp->data); temp = temp->prev; }
    printf("NULL\n");
}

```

Time Complexity – Doubly Linked List

Operation	Time Complexity
Insert at Beginning	O(1)

Operation	Time Complexity
Insert at End	$O(n)$
Delete by Value	$O(n)$
Traverse Forward/Backward	$O(n)$

1.5 Circular Singly Linked List

Priority: Tier 1 — Maximum Depth

Concept

In a circular singly linked list, every node has a next pointer and the last node's next points back to the head instead of NULL. There is no NULL terminator. The structure forms a closed loop.

Structure:

head

|

v

[10|->] -> [20|->] -> [30|->] -> [40|->]

^

|

+-----+

(last node's next = head)

Why this matters: - Traversal can start from any node and return to the same node -
No need to check for NULL during traversal (use a sentinel condition instead) - Ideal

for applications requiring continuous cycling: round-robin scheduling, circular buffers

- Polynomial multiplication benefits from circular traversal without restarting

Node Structure

```
struct CNode {  
    int data;  
    struct CNode *next;  
};
```

The difference from a singly list is purely in how the last node's next is set.

Create

```
struct CNode* createCNode(int val) {  
    struct CNode* newNode = (struct CNode*)malloc(sizeof(struct CNode));  
    newNode->data = val;  
    newNode->next = newNode; // points to itself — single-node circular list  
    return newNode;  
}
```

Single-node circular list:

```
head -> [10|+]  
      |  
      +-- points back to itself
```

Insert at Beginning

Algorithm: 1. Create a new node 2. If list is empty: `newNode->next = newNode`, `head = newNode`, return 3. Find the last node (node whose `next == head`) 4. Set `newNode->next = head` 5. Set `lastNode->next = newNode` 6. Set `head = newNode`

```

struct CNode* cInsertBeginning(struct CNode* head, int val) {
    struct CNode* newNode = (struct CNode*)malloc(sizeof(struct CNode)); // Line 1
    newNode->data = val; // Line 2
    if (head == NULL) { // Line 3
        newNode->next = newNode; // Line 4
        return newNode; // Line 5
    }
    struct CNode* last = head; // Line 6
    while (last->next != head) last = last->next; // Line 7
    newNode->next = head; // Line 8
    last->next = newNode; // Line 9
    return newNode; // Line 10
}

```

Line-by-line explanation: 1. Allocate new node 2. Store value 3. Check if list is empty 4. If empty, point new node to itself to maintain circularity 5. Return the new single node as head 6. Start at head to find the last node 7. Traverse until we find node whose next is head (that's the last node) 8. New node's next = old head 9. Last node now points to new node, completing the circle 10. New node becomes new head

 Exam Tip: The critical difference in a circular list insert-at-beginning is that you **MUST** update the last node's next to point to the new head. Forgetting this breaks the circular property. Examiners frequently ask you to trace this pointer update.

Dry Run — Insert 10, 20, 30 at Beginning:

Step 1: Insert 10. head = NULL. - newNode(10)->next = newNode(10) - head = node(10)

head -> [10|*] (* means self-loop)

Step 2: Insert 20. head = node(10). - last: traverse from head. last->next == head?
node(10)->next == node(10) == head? YES. So last = node(10). - newNode(20)->next
= head = node(10) - last->next = newNode(20) → node(10)->next = node(20) - head =
node(20)

head -> [20|->] -> [10|->]



Step 3: Insert 30. head = node(20). - last: start at node(20). node(20)->next =
node(10) != node(20) (head). Advance. node(10)->next = node(20) == head. So last =
node(10). - newNode(30)->next = head = node(20) - last->next = newNode(30) →
node(10)->next = node(30) - head = node(30)

head -> [30|->] -> [20|->] -> [10|->]



Dry Run 2 — Insert 5 at Beginning (different values):

Given list: head -> [15|->] -> [25|->] -> [35|->] (35 points back to 15)

- last: node(15)->next=node(25) != node(15)(head). node(25)->next=node(35) != head. node(35)->next=node(15) == head. last = node(35).
- newNode(5)->next = head = node(15)
- node(35)->next = node(5)
- head = node(5)

head -> [5|->] -> [15|->] -> [25|->] -> [35|->]



Insert at End

Algorithm: 1. Create new node 2. If list is empty: self-loop, return newNode 3. Traverse to find last node (node->next == head) 4. Set newNode->next = head 5. Set lastNode->next = newNode

```
struct CNode* cInsertEnd(struct CNode* head, int val) {  
    struct CNode* newNode = (struct CNode*)malloc(sizeof(struct CNode)); // Line 1  
    newNode->data = val; // Line 2  
    if (head == NULL) { // Line 3  
        newNode->next = newNode; // Line 4  
        return newNode; // Line 5  
    }  
    struct CNode* last = head; // Line 6  
    while (last->next != head) last = last->next; // Line 7  
    newNode->next = head; // Line 8  
    last->next = newNode; // Line 9  
    return head; // Line 10  
}
```

Line-by-line explanation: 1-5. Same as insertBeginning for empty list case 6. Start at head 7. Traverse to last node 8. New node's next points to head (preserves circular property) 9. Old last node now points to new node 10. Head is unchanged (we inserted at end)

Diagram — Insert 40 at End:

Before: head -> [10] -> [20] -> [30] -> (back to 10)

Traversal finds last = node(30), since node(30)->next = node(10) = head

newNode(40)->next = head = node(10)

node(30)->next = node(40)

After: head -> [10] -> [20] -> [30] -> [40] -> (back to 10)

Traversal finds last = node(30), since node(30)->next = node(10) = head

newNode(40)->next = head = node(10)

node(30)->next = node(40)

After: head -> [10] -> [20] -> [30] -> [40] -> (back to 10)

```
newNode(40)->next = head = node(10)
node(30)->next = node(40)
```

After: head -> [10] -> [20] -> [30] -> [40] -> (back to 10)

```
node(30)->next = node(40)
```

After: head -> [10] -> [20] -> [30] -> [40] -> (back to 10)

After: head -> [10] -> [20] -> [30] -> [40] -> (back to 10)

📌 Exam Tip: In insert-at-end for circular singly, the new node's next must be set to HEAD (not NULL). This is the most common exam pitfall. Circular lists NEVER contain NULL in any next pointer.

Insert at Position

[illegible]

Line-by-line explanation: 1. Delegate position-1 to insertBeginning (which handles last-node update) 2-3. Allocate and set value 4. Start at head 5-6. Move forward pos-1 steps 7. If we've looped back to head, position is out of range 8. New node's next = successor of current node 9. Current node now points to new node 10. Head unchanged

Diagram — Insert 25 at position 3:

Before: head -> [10] -> [20] -> [30] -> [40] -> (back to 10)

Position 3 means we want: ... -> [20] -> [25] -> [30] -> ...

temp starts at node(10)

i=1: temp = node(10)->next = node(20) (loop runs for i=1 to pos-2=1)

newNode(25)->next = node(20)->next = node(30)

node(20)->next = node(25)

After: head -> [10] -> [20] -> [25] -> [30] -> [40] -> (back to 10)

Traverse

Traversal must stop when we return to head, not when next is NULL (there is no NULL).

```
void cTraverse(struct CNode* head) {  
    if (head == NULL) { printf("Empty list\n"); return; } // Line 1  
    struct CNode* temp = head;                          // Line 2  
    do {                                                  // Line 3  
        printf("%d -> ", temp->data);                    // Line 4  
        temp = temp->next;                                // Line 5  
    } while (temp->next != head);  
}
```

```
    } while (temp != head);           // Line 6

    printf("(back to head)\n");       // Line 7
}
```

Line-by-line explanation: 1. Guard for empty list 2. Start at head 3. Use do-while so head is printed before condition is checked 4. Print current node's data 5. Move to next node 6. Stop when we've come full circle back to head 7. Print circular indicator

📌 Exam Tip: The do-while loop for traversal is the canonical pattern for circular lists. A regular while loop with `temp != head` would skip the head node entirely. Examiners check whether you use do-while here.

Dry Run – Traverse [10 -> 20 -> 30 -> back to 10]:

Step 1: temp = node(10). Print 10. temp = node(20). temp != head(10)? YES continue.

Step 2: temp = node(20). Print 20. temp = node(30). temp != head? YES continue.

Step 3: temp = node(30). Print 30. temp = node(10). temp != head? NO. Stop.

Output: 10 -> 20 -> 30 -> (back to head)

Delete by Value

Algorithm: 1. Handle empty list 2. If head is the node to delete: find last node, update last->next = head->next, set head = head->next, free old head 3. Otherwise: find the node just before target, update its next, free target

```
struct CNode* cDeleteByValue(struct CNode* head, int val) {  
    if (head == NULL) { printf("Empty list\n"); return NULL; } // Line 1  
  
    struct CNode* curr = head; // Line 2  
  
    struct CNode* prev = NULL; // Line 3  
  
    // Find the node to delete and its predecessor  
    do { // Line 4
```

```

    if (curr->data == val) break;                // Line 5
    prev = curr;                                // Line 6
    curr = curr->next;                            // Line 7
} while (curr != head);                        // Line 8
if (curr->data != val) { printf("Not found\n"); return head; } // Line 9
// Deleting the only node
if (curr == head && curr->next == head) {        // Line 10
    free(curr);                                // Line 11
    return NULL;                               // Line 12
}
// Deleting head node
if (curr == head) {                            // Line 13
    struct CNode* last = head;                 // Line 14
    while (last->next != head) last = last->next; // Line 15
    last->next = head->next;                    // Line 16
    head = head->next;                          // Line 17
    free(curr);                                // Line 18
    return head;                               // Line 19
}
// Deleting non-head node
prev->next = curr->next;                        // Line 20
free(curr);                                   // Line 21
return head;                                  // Line 22
}

```

Line-by-line explanation: 1. Empty list guard 2. curr starts at head 3. prev tracks the node before curr 4. do-while to include head in search 5. Found the target — break 6. Advance prev 7. Advance curr 8. Stop if we've gone full circle 9. If curr came back to

head without match, value not in list 10. Check if this is the only node (self-loop) 11-12. Free it, return NULL (empty list) 13. Check if deleting the head 14-15. Find last node 16. Last node's next bypasses old head 17. Head advances to next node 18. Free old head 19. Return new head 20. Bypass the target node (non-head case) 21. Free target 22. Return head

 Exam Tip: Deletion of the HEAD node in a circular list is the most tested scenario. You MUST find the last node and update its next pointer. Examiners trace through this step specifically. Three cases: (a) empty, (b) single node, (c) deleting head, (d) deleting non-head — cover all four for full marks.

Dry Run — Delete 20 from [10 -> 20 -> 30 -> back to 10]:

head -> [10|->] -> [20|->] -> [30|->]

|
(30->next = head = 10)

Step 1: curr = node(10), prev = NULL. node(10)->data == 20? NO. prev=node(10), curr=node(20).

Step 2: curr = node(20). node(20)->data == 20? YES. Break.

Step 3: curr != head, so we are in the non-head deletion case (Line 20).

Step 4: prev->next = curr->next → node(10)->next = node(30)

Step 5: free(node(20))

After: head -> [10|->] -> [30|->]

|
(30->next = head = 10)

Dry Run — Delete 10 (head) from [10 -> 20 -> 30 -> back to 10]:

Step 1: curr=node(10), data==10, break immediately.

Step 2: curr == head, not single node.

Step 3: Find last: last = node(10). node(10)->next = node(20) != head?

last = node(20). node(20)->next = node(30) != head?

last = node(30). node(30)->next = node(10) == head. last = node(30).

Step 4: last->next = head->next → node(30)->next = node(20)

Step 5: head = node(20)

Step 6: free(node(10))

After: head -> [20|->] -> [30|->]

|
(30->next = head = 20)

Delete by Position

```
struct CNode* cDeleteAtPos(struct CNode* head, int pos) {
    if (head == NULL) return NULL;
    if (pos == 1) return cDeleteByValue(head, head->data);
    struct CNode* temp = head;
    for (int i = 1; i < pos - 1; i++) {
        temp = temp->next;
        if (temp == head) { printf("Invalid position\n"); return head; }
    }
    struct CNode* toDelete = temp->next;
    if (toDelete == head) { printf("Invalid position\n"); return head; }
    temp->next = toDelete->next;
    free(toDelete);
    return head;
}
```

Search

```
int cSearch(struct CNode* head, int val) {
    if (head == NULL) return -1;
    struct CNode* temp = head;
    int pos = 1;
    do {
        if (temp->data == val) return pos;
        temp = temp->next;
        pos++;
    } while (temp != head);
    return -1;
}
```

Update

```
void cUpdate(struct CNode* head, int oldVal, int newVal) {
    if (head == NULL) return;
    struct CNode* temp = head;
    do {
        if (temp->data == oldVal) { temp->data = newVal; return; }
        temp = temp->next;
    } while (temp != head);
    printf("Value not found\n");
}
```

Complete Program – Circular Singly Linked List

```
#include <stdio.h>

#include <stdlib.h>
```

```
struct CNode {  
    int data;  
    struct CNode *next;  
};
```

```
struct CNode* createCNode(int val) {  
    struct CNode* n = (struct CNode*)malloc(sizeof(struct CNode));  
    n->data = val;  
    n->next = n;  
    return n;  
}
```

```
struct CNode* cInsertEnd(struct CNode* head, int val) {  
    struct CNode* newNode = createCNode(val);  
    if (head == NULL) return newNode;  
    struct CNode* last = head;  
    while (last->next != head) last = last->next;  
    newNode->next = head;  
    last->next = newNode;  
    return head;  
}
```

```
struct CNode* cInsertBeginning(struct CNode* head, int val) {  
    struct CNode* newNode = createCNode(val);  
    if (head == NULL) return newNode;  
    struct CNode* last = head;  
    while (last->next != head) last = last->next;
```

```
newNode->next = head;

last->next = newNode;

return newNode;
}
```

```
void cTraverse(struct CNode* head) {
    if (head == NULL) { printf("Empty\n"); return; }
    struct CNode* temp = head;
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(head)\n");
}
```

```
struct CNode* cDelete(struct CNode* head, int val) {
    if (head == NULL) return NULL;
    struct CNode* curr = head, *prev = NULL;
    do {
        if (curr->data == val) break;
        prev = curr;
        curr = curr->next;
    } while (curr != head);
    if (curr->data != val) { printf("Not found\n"); return head; }
    if (curr == head && curr->next == head) { free(curr); return NULL; }
    if (curr == head) {
        struct CNode* last = head;
        while (last->next != head) last = last->next;
```

```

        last->next = head->next;

        head = head->next;

        free(curr);

        return head;
    }

    prev->next = curr->next;

    free(curr);

    return head;
}

int main() {

    struct CNode* head = NULL;

    head = cInsertEnd(head, 10);
    head = cInsertEnd(head, 20);
    head = cInsertEnd(head, 30);
    head = cInsertEnd(head, 40);

    printf("After inserting 10,20,30,40 at end: ");
    cTraverse(head);

    head = cInsertBeginning(head, 5);
    printf("After inserting 5 at beginning: ");
    cTraverse(head);

    head = cDelete(head, 20);
    printf("After deleting 20: ");
    cTraverse(head);

    head = cDelete(head, 5);
    printf("After deleting head (5): ");
    cTraverse(head);
}

```

```
    return 0;
}
```

Expected Output:

After inserting 10,20,30,40 at end: 10 -> 20 -> 30 -> 40 -> (head)

After inserting 5 at beginning: 5 -> 10 -> 20 -> 30 -> 40 -> (head)

After deleting 20: 5 -> 10 -> 30 -> 40 -> (head)

After deleting head (5): 10 -> 30 -> 40 -> (head)

Time Complexity – Circular Singly Linked List

Operation	Time	Space
Insert Beginning	$O(n)$ – must find last node	$O(1)$
Insert End	$O(n)$ – must traverse to last	$O(1)$
Insert at Position k	$O(k)$	$O(1)$
Delete (any)	$O(n)$	$O(1)$
Traverse	$O(n)$	$O(1)$
Search	$O(n)$	$O(1)$

Note: Insert at beginning is $O(n)$ unlike a singly linked list because we must update the last node's pointer. If you maintain a tail pointer, insert-beginning and insert-end both become $O(1)$.

5-Mark Answer Blueprint – Circular Singly Linked List

For a perfect 5-mark answer on CSLL operations, include:

- **Definition** (1 sentence): A circular singly linked list is a singly linked list where the last node's next pointer points back to the head node, forming a closed loop with no NULL terminator.
 - **Node structure** (code, 4 lines): Show the struct with data and *next.
 - **The operation requested** (algorithm + code): Write the specific operation asked (insert/delete/traverse). Include the do-while loop for traversal, or the last-node-finding loop for insert/delete.
 - **Critical pointer update** (must state): For any modification, explicitly state that the last node's next must always point to head.
 - **Dry run on example** (3-4 nodes): Trace through 3-4 steps showing exact pointer states.
 - **Complexity**: $O(n)$ for most operations, $O(1)$ if tail pointer is maintained.
-

1.6 Circular Doubly Linked List

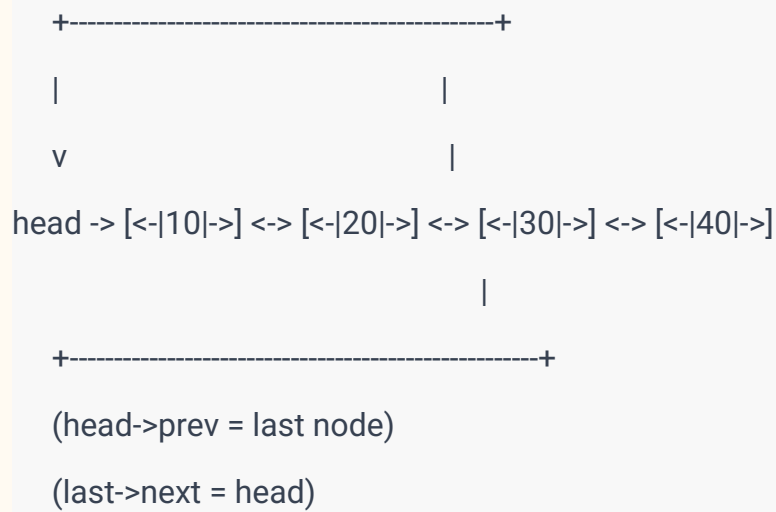
Priority: Tier 1 — Maximum Depth

Concept

A circular doubly linked list combines the bidirectionality of a doubly linked list with the circular property. Every node has:

- A next pointer pointing to the successor node
- A prev pointer pointing to the predecessor node
- The last node's next points to head
- The head node's prev points to the last node

Structure:

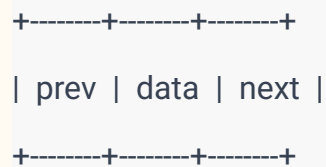


This structure allows: - $O(1)$ access to BOTH the head and tail (tail = head->prev) - Bidirectional traversal in a circular manner - Efficient insert/delete at both ends

Node Structure

```
struct CDNode {
    int data;
    struct CDNode *prev;
    struct CDNode *next;
};
```

Node layout:



Create (Single Node)

```
struct CDNode* createCDNode(int val) {
    struct CDNode* newNode = (struct CDNode*)malloc(sizeof(struct CDNode));
    newNode->data = val;
```

```

newNode->next = newNode; // points to itself
newNode->prev = newNode; // points to itself
return newNode;
}

```

Single-node state:

```

head -> [ <-|10|-> ]
      | |
      +--+ (both prev and next point to self)

```

Insert at Beginning

Algorithm: 1. Create new node 2. If empty: self-loop both pointers, return 3. Find last node (last = head->prev) 4. newNode->next = head 5. newNode->prev = last 6. head->prev = newNode 7. last->next = newNode 8. head = newNode

```

struct CDNode* cdInsertBeginning(struct CDNode* head, int val) {
    struct CDNode* newNode = createCDNode(val); // Line 1
    if (head == NULL) return newNode;           // Line 2
    struct CDNode* last = head->prev;           // Line 3
    newNode->next = head;                        // Line 4
    newNode->prev = last;                        // Line 5
    head->prev = newNode;                        // Line 6
    last->next = newNode;                        // Line 7
    return newNode;                             // Line 8 — new head
}

```

Line-by-line explanation: 1. Create new node (self-pointing initially) 2. If list empty, single self-pointing node is the full list 3. Since last->next = head always, we can get last in O(1) using head->prev 4. New node's successor is the old head 5. New node's

predecessor is the last node (it's circular, so new head's prev is last) 6. Old head's predecessor is now the new node 7. Last node's next was old head, now points to new node 8. New node becomes the head

 Exam Tip: The key insight is $\text{last} = \text{head} \rightarrow \text{prev}$ — in a CDLL you get the last node in $O(1)$ without traversal. This is more efficient than CSLL. Examiners specifically award marks for using this shortcut correctly.

Dry Run — Insert 10, 20, 30 at Beginning:

Step 1: Insert 10, head = NULL. - $\text{newNode}(10) \rightarrow \text{next} = \text{newNode}(10)$, $\rightarrow \text{prev} = \text{newNode}(10)$ - head = node(10)

head -> [node10: prev=node10, data=10, next=node10]

Step 2: Insert 20. head = node(10). - last = head->prev = node(10) (only one node, so last = head) - $\text{newNode}(20) \rightarrow \text{next} = \text{node}(10)$ - $\text{newNode}(20) \rightarrow \text{prev} = \text{node}(10)$ - node(10)->prev = node(20) [old head's prev now points to new node] - node(10)->next = node(20) [last node's next now points to new node] - head = node(20)

head -> [node20: prev=node10, data=20, next=node10]

[node10: prev=node20, data=10, next=node20]

(circular, 2 nodes)

Step 3: Insert 30. head = node(20). - last = head->prev = node(10) - $\text{newNode}(30) \rightarrow \text{next} = \text{node}(20)$ - $\text{newNode}(30) \rightarrow \text{prev} = \text{node}(10)$ - node(20)->prev = node(30) - node(10)->next = node(30) - head = node(30)

head -> [30: prev=10, next=20] <-> [20: prev=30, next=10] <-> [10: prev=20, next=30]

circular

Insert at End

Algorithm: 1. Create new node 2. If empty, return self-pointing new node 3. last = head->prev 4. newNode->prev = last 5. newNode->next = head 6. last->next = newNode 7. head->prev = newNode 8. head unchanged

```
struct CDNode* cdInsertEnd(struct CDNode* head, int val) {  
    struct CDNode* newNode = createCDNode(val);    // Line 1  
    if (head == NULL) return newNode;              // Line 2  
    struct CDNode* last = head->prev;              // Line 3  
    newNode->prev = last;                          // Line 4  
    newNode->next = head;                          // Line 5  
    last->next = newNode;                          // Line 6  
    head->prev = newNode;                          // Line 7  
    return head;                                   // Line 8  
}
```

Line-by-line explanation: 1. Allocate new node 2. Empty case — single self-pointing node 3. Get last node in $O(1)$ from head->prev 4. New node's predecessor is the old last 5. New node's successor is head (circular) 6. Old last now points forward to new node 7. Head's prev now points back to new last node 8. Head doesn't change

Diagram — Insert 40 at End to [10 <-> 20 <-> 30]:

Before:

head(10)->prev = node(30)

node(30)->next = head(10)

After cdInsertEnd(head, 40):

last = node(30)

```

newNode(40)->prev = node(30)
newNode(40)->next = head(10)
node(30)->next = node(40)
node(10)->prev = node(40)

```

After:

head -> [10] <-> [20] <-> [30] <-> [40] (circular)

```

      ^           |
      +-----+ (40->next = 10, 10->prev = 40)

```

 Exam Tip: Insert-at-end in CDLL requires exactly 4 pointer updates: newNode->prev, newNode->next, last->next, head->prev. Any fewer and the circular bidirectional property is broken. Full marks require showing all four updates.

Insert at Position

```

struct CDNode* cdInsertAtPos(struct CDNode* head, int val, int pos) {
    if (pos == 1) return cdInsertBeginning(head, val);    // Line 1
    struct CDNode* newNode = (struct CDNode*)malloc(sizeof(struct CDNode)); //
Line 2
    newNode->data = val;                                   // Line 3
    struct CDNode* temp = head;                           // Line 4
    for (int i = 1; i < pos - 1; i++) {                   // Line 5
        temp = temp->next;                                 // Line 6
        if (temp == head) { printf("Out of range\n"); return head; } // Line 7
    }
    struct CDNode* nextNode = temp->next;                 // Line 8
    newNode->next = nextNode;                             // Line 9

```

```

newNode->prev = temp;                // Line 10
temp->next = newNode;                // Line 11
nextNode->prev = newNode;            // Line 12
return head;                        // Line 13
}

```

Line-by-line: 1. Position 1 delegates to insertBeginning 2-3. Allocate and set value 4. Start at head 5-6. Move to node at position pos-1 7. If we've gone full circle, position is invalid 8. Store the node currently at position pos 9. New node's next = old node at pos 10. New node's prev = node at pos-1 11. Node at pos-1 now points forward to new node 12. Old node at pos now points back to new node 13. Head unchanged

Diagram — Insert 25 at pos 3 in [10 <-> 20 <-> 30 <-> 40]:

temp reaches node(20) after 1 iteration (pos-1 = 2, so loop runs once)

nextNode = node(20)->next = node(30)

newNode(25)->next = node(30)

newNode(25)->prev = node(20)

node(20)->next = node(25)

node(30)->prev = node(25)

Result: [10] <-> [20] <-> [25] <-> [30] <-> [40] (circular)

Traverse Forward and Backward

```

void cdTraverseForward(struct CDNode* head) {
    if (head == NULL) { printf("Empty\n"); return; }
    struct CDNode* temp = head;
    do {

```

```

        printf("%d <-> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(head)\n");
}

void cdTraverseBackward(struct CDNode* head) {
    if (head == NULL) { printf("Empty\n"); return; }
    struct CDNode* temp = head->prev; // start from last node
    do {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    } while (temp != head->prev);
    printf("(tail)\n");
}

```

Dry Run — Forward Traverse [10 <-> 20 <-> 30]:

```

do: temp=node(10), print 10, temp=node(20). temp != head(10)? YES.
do: temp=node(20), print 20, temp=node(30). temp != head? YES.
do: temp=node(30), print 30, temp=node(10). temp != head? NO. Stop.
Output: 10 <-> 20 <-> 30 <-> (head)

```

Delete by Value

Algorithm: 1. Handle empty list 2. Search for node with matching value using do-while 3. If only one node, free and return NULL 4. Update predecessor's next and successor's prev 5. If deleting head, update head

```

struct CDNode* cdDeleteByValue(struct CDNode* head, int val) {
    if (head == NULL) { printf("Empty list\n"); return NULL; } // Line 1
    struct CDNode* curr = head; // Line 2
    do { // Line 3
        if (curr->data == val) break; // Line 4
        curr = curr->next; // Line 5
    } while (curr != head); // Line 6
    if (curr->data != val) { printf("Not found\n"); return head; } // Line 7
    if (curr->next == curr) { free(curr); return NULL; } // Line 8
    curr->prev->next = curr->next; // Line 9
    curr->next->prev = curr->prev; // Line 10
    struct CDNode* newHead = (curr == head) ? head->next : head; // Line 11
    free(curr); // Line 12
    return newHead; // Line 13
}

```

Line-by-line explanation: 1. Guard: empty list returns NULL 2. Start search at head 3-6. do-while traversal finds the target or circles back 7. If we looped without finding, report not found 8. Single-node case: free and return empty list 9. Predecessor's next now skips curr, pointing to curr's successor 10. Successor's prev now skips curr, pointing to curr's predecessor 11. If we're deleting head, new head is head->next; otherwise head unchanged 12. Free the deleted node 13. Return (possibly updated) head

 Exam Tip: Lines 9 and 10 are the two symmetric pointer updates that define CDLL deletion. Both must always be present. Line 9 repairs the forward chain; Line 10 repairs the backward chain. Missing either one is a common 1-mark deduction.

Dry Run — Delete 20 from [10 <-> 20 <-> 30 <-> 40] (circular):

Search: curr=node(10), data!=20, curr=node(20), data==20. Break.

Not single node.

node(20)->prev = node(10). node(10)->next = node(20)->next = node(30).

node(20)->next = node(30). node(30)->prev = node(20)->prev = node(10).

curr != head, so newHead = head = node(10).

free(node(20)).

Before: [10] <-> [20] <-> [30] <-> [40]

After: [10] <-> [30] <-> [40] (circular, 10->prev=40, 40->next=10)

Dry Run — Delete 10 (head) from [10 <-> 20 <-> 30]:

Search: curr=node(10), data==10. Break immediately.

Not single node.

node(10)->prev = node(30). node(30)->next = node(10)->next = node(20).

node(10)->next = node(20). node(20)->prev = node(10)->prev = node(30).

curr == head, so newHead = head->next = node(20).

free(node(10)).

After: head = node(20). [20] <-> [30] (circular, 20->prev=30, 30->next=20)

Complete Program — Circular Doubly Linked List

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct CDNode {
```

```
    int data;
```

```

    struct CDNode *prev, *next;
};

struct CDNode* createCDNode(int val) {
    struct CDNode* n = (struct CDNode*)malloc(sizeof(struct CDNode));
    n->data = val; n->prev = n; n->next = n;
    return n;
}

struct CDNode* cdInsertEnd(struct CDNode* head, int val) {
    struct CDNode* newNode = createCDNode(val);
    if (head == NULL) return newNode;
    struct CDNode* last = head->prev;
    newNode->prev = last; newNode->next = head;
    last->next = newNode; head->prev = newNode;
    return head;
}

struct CDNode* cdInsertBeginning(struct CDNode* head, int val) {
    struct CDNode* newNode = createCDNode(val);
    if (head == NULL) return newNode;
    struct CDNode* last = head->prev;
    newNode->next = head; newNode->prev = last;
    head->prev = newNode; last->next = newNode;
    return newNode;
}

void cdTraverse(struct CDNode* head) {

```

```

    if (!head) { printf("Empty\n"); return; }

    struct CDNode* temp = head;

    do { printf("%d <-> ", temp->data); temp = temp->next; } while (temp != head);

    printf("(head)\n");
}

```

```

struct CDNode* cdDelete(struct CDNode* head, int val) {
    if (!head) return NULL;

    struct CDNode* curr = head;

    do { if (curr->data == val) break; curr = curr->next; } while (curr != head);

    if (curr->data != val) { printf("Not found\n"); return head; }

    if (curr->next == curr) { free(curr); return NULL; }

    curr->prev->next = curr->next;
    curr->next->prev = curr->prev;

    struct CDNode* newHead = (curr == head) ? head->next : head;

    free(curr);

    return newHead;
}

```

```

int main() {
    struct CDNode* head = NULL;

    head = cdInsertEnd(head, 10);
    head = cdInsertEnd(head, 20);
    head = cdInsertEnd(head, 30);
    head = cdInsertEnd(head, 40);

    printf("List: "); cdTraverse(head);

    head = cdInsertBeginning(head, 5);

    printf("After insert 5 at beginning: "); cdTraverse(head);
}

```

```

    head = cdDelete(head, 20);
    printf("After delete 20: "); cdTraverse(head);
    head = cdDelete(head, 5);
    printf("After delete head (5): "); cdTraverse(head);
    return 0;
}

```

Expected Output:

List: 10 <-> 20 <-> 30 <-> 40 <-> (head)

After insert 5 at beginning: 5 <-> 10 <-> 20 <-> 30 <-> 40 <-> (head)

After delete 20: 5 <-> 10 <-> 30 <-> 40 <-> (head)

After delete head (5): 10 <-> 30 <-> 40 <-> (head)

Time Complexity – Circular Doubly Linked List

Operation	Time	Note
Insert Beginning	$O(1)$	last = head->prev, no traversal
Insert End	$O(1)$	last = head->prev, no traversal
Insert at Position k	$O(k)$	Traverse k steps
Delete	$O(n)$	Must search for value
Traverse	$O(n)$	Visit all nodes

CDLL is the only linked list variant where both insert-at-beginning AND insert-at-end are $O(1)$, because head->prev gives instant access to the last node.

5-Mark Answer Blueprint – Circular Doubly Linked List

- **Definition:** A CDLL is a doubly linked list where the last node's next points to head and head's prev points to the last node, enabling bidirectional circular traversal.
 - **Node structure:** Show struct with prev, data, next.
 - **Key invariants:** Always state: (a) last->next = head, (b) head->prev = last, (c) no NULL pointers anywhere.
 - **Requested operation:** Write algorithm and code. Show all four pointer updates for insert-at-end (newNode->prev, newNode->next, last->next, head->prev).
 - **Dry run:** 3-4 node example showing pointer states before and after.
 - **Complexity:** $O(1)$ for begin/end inserts (unique to CDLL), $O(n)$ for search/delete.
-

1.7 Polynomial Representation using Singly Linked List

Priority: Tier 2

Why Linked Lists for Polynomials?

A polynomial like $P(x) = 8x^{100} + 3x^5 + 2$ has 3 non-zero terms but degree 100. An array representation would need 101 slots, 98 of which store 0. This wastes memory. A sparse polynomial is best represented as a linked list where each node stores only a non-zero term.

Node Structure

```
struct PolyNode {  
    int coeff; // coefficient of the term  
    int exp;   // exponent (power of x)  
    struct PolyNode *next;  
};
```

Polynomial: $5x^4 + 3x^2 + 2x + 7$

Linked List:

head -> [5|4|->] -> [3|2|->] -> [2|1|->] -> [7|0|NULL]
 coeff exp coeff exp

Create Polynomial

Read terms in decreasing order of exponent and insert at end:

```
struct PolyNode* createPoly() {  
    int n, coeff, exp;  
    printf("Enter number of terms: ");  
    scanf("%d", &n);  
    struct PolyNode* head = NULL;  
    struct PolyNode* last = NULL;  
    for (int i = 0; i < n; i++) {  
        printf("Enter coeff and exp: ");  
        scanf("%d %d", &coeff, &exp);  
        struct PolyNode* newNode = (struct PolyNode*)malloc(sizeof(struct  
PolyNode));  
        newNode->coeff = coeff;
```

```

newNode->exp = exp;
newNode->next = NULL;
if (head == NULL) { head = newNode; last = newNode; }
else { last->next = newNode; last = newNode; }
}
return head;
}

```

Display Polynomial

```

void displayPoly(struct PolyNode* head) {
    struct PolyNode* temp = head;
    while (temp != NULL) {
        if (temp->exp == 0)    printf("%d", temp->coeff);
        else if (temp->exp == 1) printf("%dx", temp->coeff);
        else                  printf("%dx^%d", temp->coeff, temp->exp);
        if (temp->next != NULL) printf(" + ");
        temp = temp->next;
    }
    printf("\n");
}

```

Polynomial Addition using Singly Linked List

Algorithm (merge with exponent comparison):

- Set p1 = head1, p2 = head2, result = NULL
- While both p1 and p2 are not NULL:
 - Case A: p1->exp > p2->exp → copy p1 term to result, advance p1
 - Case B: p1->exp < p2->exp → copy p2 term to result, advance p2

- Case C: $p1 \rightarrow \text{exp} == p2 \rightarrow \text{exp} \rightarrow$ add coefficients, create new term if sum $\neq 0$, advance both
- Copy remaining terms from whichever list is non-empty
- Return result

```

struct PolyNode* addPoly(struct PolyNode* p1, struct PolyNode* p2) {
    struct PolyNode* result = NULL;
    struct PolyNode* last = NULL;

    while (p1 != NULL && p2 != NULL) {
        struct PolyNode* newNode = (struct PolyNode*)malloc(sizeof(struct
PolyNode));
        newNode->next = NULL;

        if (p1->exp > p2->exp) {                // Case A
            newNode->coeff = p1->coeff;
            newNode->exp = p1->exp;
            p1 = p1->next;
        } else if (p1->exp < p2->exp) {          // Case B
            newNode->coeff = p2->coeff;
            newNode->exp = p2->exp;
            p2 = p2->next;
        } else {                                // Case C
            int sum = p1->coeff + p2->coeff;
            if (sum == 0) { free(newNode); p1 = p1->next; p2 = p2->next; continue; }
            newNode->coeff = sum;
            newNode->exp = p1->exp;
            p1 = p1->next; p2 = p2->next;
        }

        if (result == NULL)
            result = newNode;
        else
            last->next = newNode;
        last = newNode;
    }

    if (p1 != NULL)
        while (p1 != NULL) {
            struct PolyNode* temp = (struct PolyNode*)malloc(sizeof(struct
PolyNode));
            temp->coeff = p1->coeff;
            temp->exp = p1->exp;
            temp->next = NULL;
            last->next = temp;
            last = temp;
            p1 = p1->next;
        }
    if (p2 != NULL)
        while (p2 != NULL) {
            struct PolyNode* temp = (struct PolyNode*)malloc(sizeof(struct
PolyNode));
            temp->coeff = p2->coeff;
            temp->exp = p2->exp;
            temp->next = NULL;
            last->next = temp;
            last = temp;
            p2 = p2->next;
        }

    return result;
}

```

```

    }

    if (result == NULL) { result = newNode; last = newNode; }
    else { last->next = newNode; last = newNode; }
}

while (p1 != NULL) { /* copy remaining p1 terms */
    struct PolyNode* newNode = (struct PolyNode*)malloc(sizeof(struct
PolyNode));

    newNode->coeff = p1->coeff; newNode->exp = p1->exp; newNode->next = NULL;
    if (result == NULL) { result = newNode; last = newNode; }
    else { last->next = newNode; last = newNode; }
    p1 = p1->next;
}

while (p2 != NULL) { /* copy remaining p2 terms */
    struct PolyNode* newNode = (struct PolyNode*)malloc(sizeof(struct
PolyNode));

    newNode->coeff = p2->coeff; newNode->exp = p2->exp; newNode->next = NULL;
    if (result == NULL) { result = newNode; last = newNode; }
    else { last->next = newNode; last = newNode; }
    p2 = p2->next;
}

return result;
}

```

Dry Run — Add $P1 = 5x^4 + 3x^2 + 2x + 7$ and $P2 = 4x^3 + 3x^2 + x + 1$:

P1: [5|4] -> [3|2] -> [2|1] -> [7|0]

P2: [4|3] -> [3|2] -> [1|1] -> [1|0]

Step 1: $p1 \rightarrow \text{exp}=4$, $p2 \rightarrow \text{exp}=3$. $4 > 3 \rightarrow$ Case A. result: [5|4]. p1 advances.

p1=[3|2], p2=[4|3]

Step 2: p1->exp=2, p2->exp=3. $2 < 3 \rightarrow$ Case B. result: [5|4]->[4|3]. p2 advances.

p1=[3|2], p2=[3|2]

Step 3: p1->exp=2, p2->exp=2. Equal $\rightarrow$ Case C. sum=3+3=6. result: [5|4]->[4|3]->[6|2].

p1=[2|1], p2=[1|1]

Step 4: p1->exp=1, p2->exp=1. Equal $\rightarrow$ Case C. sum=2+1=3. result: ...->[3|1].

p1=[7|0], p2=[1|0]

Step 5: p1->exp=0, p2->exp=0. Equal $\rightarrow$ Case C. sum=7+1=8. result: ...->[8|0].

p1=NULL, p2=NULL. Loop ends.

Final result: $5x^4 + 4x^3 + 6x^2 + 3x + 8$

Polynomial Multiplication using Singly Linked List

Algorithm: 1. For each term T1 in P1: - For each term T2 in P2: - Compute newCoeff = T1.coeff * T2.coeff - Compute newExp = T1.exp + T2.exp - Insert (newCoeff, newExp) into result list - If a term with same exponent exists in result, add to its coefficient 2. Return result list (terms sorted by decreasing exponent)

```
struct PolyNode* insertOrAdd(struct PolyNode* result, int coeff, int exp) {  
    // Insert into result maintaining sorted order; add if exponent exists  
    struct PolyNode* temp = result;  
    while (temp != NULL) {  
        if (temp->exp == exp) { temp->coeff += coeff; return result; }  
        temp = temp->next;  
    }  
    // If no term with same exponent exists, create new node and add to result  
    struct PolyNode* newNode = (struct PolyNode*) malloc(sizeof(struct PolyNode));  
    newNode->coeff = coeff;  
    newNode->exp = exp;  
    newNode->next = NULL;  
    if (result == NULL) return newNode;  
    struct PolyNode* prev = NULL;  
    while (temp != NULL) {  
        if (temp->exp < exp) {  
            if (prev == NULL) {  
                result = newNode;  
            } else {  
                prev->next = newNode;  
            }  
            prev = temp;  
        }  
        temp = temp->next;  
    }  
    prev->next = newNode;  
    return result;  
}
```

```

}

// Not found, insert in sorted position
struct PolyNode* newNode = (struct PolyNode*)malloc(sizeof(struct PolyNode));
newNode->coeff = coeff; newNode->exp = exp; newNode->next = NULL;
if (result == NULL || result->exp < exp) { newNode->next = result; return newNode; }
temp = result;
while (temp->next != NULL && temp->next->exp > exp) temp = temp->next;
newNode->next = temp->next; temp->next = newNode;
return result;
}

struct PolyNode* multiplyPoly(struct PolyNode* p1, struct PolyNode* p2) {
    struct PolyNode* result = NULL;
    for (struct PolyNode* t1 = p1; t1 != NULL; t1 = t1->next) {
        for (struct PolyNode* t2 = p2; t2 != NULL; t2 = t2->next) {
            result = insertOrAdd(result, t1->coeff * t2->coeff, t1->exp + t2->exp);
        }
    }
    return result;
}

```

Dry Run — Multiply $P1 = 3x^2 + 2x + 1$ by $P2 = x + 2$:

P1: [3|2] -> [2|1] -> [1|0]

P2: [1|1] -> [2|0]

t1=node(3,2), t2=node(1,1): $3*1=3$, $2+1=3$. result: [3|3]

t1=node(3,2), t2=node(2,0): $3*2=6$, $2+0=2$. result: [3|3]->[6|2]

t1=node(2,1), t2=node(1,1): $2*1=2$, $1+1=2$. exp=2 exists! $6+2=8$. result: [3|3]->[8|2]

t1=node(2,1), t2=node(2,0): $2*2=4$, $1+0=1$. result: [3|3]->[8|2]->[4|1]

t1=node(1,0), t2=node(1,1): $1*1=1$, $0+1=1$. exp=1 exists! $4+1=5$. result:
[3|3]->[8|2]->[5|1]

t1=node(1,0), t2=node(2,0): $1*2=2$, $0+0=0$. result: [3|3]->[8|2]->[5|1]->[2|0]

Final: $3x^3 + 8x^2 + 5x + 2$

Verify: $(3x^2 + 2x + 1)(x + 2) = 3x^3 + 6x^2 + 2x^2 + 4x + x + 2 = 3x^3 + 8x^2 + 5x + 2$

✓

1.8 Polynomial Operations using Circular Linked List

Priority: Tier 1 — Maximum Depth

Why Circular Linked List for Polynomials?

In a standard singly linked list polynomial representation, multiplying two polynomials requires:

- Iterating through all terms of P2 for every term of P1
- Restarting from P2's head each inner iteration
- Constant NULL checks to detect list end

A circular singly linked list eliminates the need to restart from NULL. The traversal is continuous. Using a sentinel node (a dummy node with exp = -1 at the head, whose next is the first real term, and whose next-to-last is the dummy node again) makes the loop termination clean and uniform.

However, for exam purposes, the most common approach is simply using a circular list where you traverse using `do { ... } while (curr != head)`.

Circular Polynomial Node Structure

```
struct CPolyNode {  
    int coeff;  
    int exp;  
    struct CPolyNode *next;  
};
```

ASCII Diagram — $P(x) = 5x^4 + 3x^2 + 2x + 7$ as Circular Singly Linked List

```
head  
|  
v  
[5|4|->] -> [3|2|->] -> [2|1|->] -> [7|0|->]  
^               |  
+-----+  
(last node's next = head)
```

Polynomial Addition using Circular Singly Linked List

Algorithm: 1. Create an empty result circular list with a dummy tail pointer 2. Set $curr1 = head1$, $curr2 = head2$ 3. Use a do-while loop to compare terms: - Case A: $curr1 \rightarrow exp > curr2 \rightarrow exp$: copy $curr1$ to result, advance $curr1$ - Case B: $curr1 \rightarrow exp < curr2 \rightarrow exp$: copy $curr2$ to result, advance $curr2$ - Case C: equal exponents: add coefficients, advance both 4. When one list is exhausted ($curr$ reaches its head), append remaining terms from the other 5. Close the result into a circular list

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

struct CPolyNode {
    int coeff, exp;
    struct CPolyNode *next;
};

// Create a polynomial node and add to circular list (inserts at end)
struct CPolyNode* cpAddTerm(struct CPolyNode* head, int coeff, int exp) {
    struct CPolyNode* newNode = (struct CPolyNode*)malloc(sizeof(struct
CPolyNode));
    newNode->coeff = coeff;
    newNode->exp = exp;
    if (head == NULL) {
        newNode->next = newNode;
        return newNode;
    }
    struct CPolyNode* last = head;
    while (last->next != head) last = last->next;
    newNode->next = head;
    last->next = newNode;
    return head;
}

// Display circular polynomial
void cpDisplay(struct CPolyNode* head) {
    if (!head) { printf("0\n"); return; }
    struct CPolyNode* curr = head;
    do {
        if (curr->exp == 0)    printf("%d", curr->coeff);

```

```

else if (curr->exp == 1) printf("%dx", curr->coeff);
else          printf("%dx^%d", curr->coeff, curr->exp);
curr = curr->next;
if (curr != head)    printf(" + ");
} while (curr != head);
printf("\n");
}

```

// Polynomial Addition using Circular Singly Linked List

```

struct CPolyNode* cpAdd(struct CPolyNode* head1, struct CPolyNode* head2) {
    struct CPolyNode* result = NULL;    // Line 1
    struct CPolyNode* c1 = head1;       // Line 2
    struct CPolyNode* c2 = head2;       // Line 3
    int done1 = 0, done2 = 0;           // Line 4: flags to track if list is exhausted

    while (!done1 && !done2) {          // Line 5
        if (c1->exp > c2->exp) {         // Line 6: Case A
            result = cpAddTerm(result, c1->coeff, c1->exp); // Line 7
            c1 = c1->next;               // Line 8
        } else if (c1->exp < c2->exp) {   // Line 9: Case B
            result = cpAddTerm(result, c2->coeff, c2->exp); // Line 10
            c2 = c2->next;               // Line 11
        } else {                        // Line 12: Case C
            int sum = c1->coeff + c2->coeff; // Line 13
            if (sum != 0)
                result = cpAddTerm(result, sum, c1->exp); // Line 14
            c1 = c1->next;               // Line 15
            c2 = c2->next;               // Line 16
        }
    }
}

```

```

    }
    if (c1 == head1) done1 = 1;    // Line 17: c1 has looped back to start
    if (c2 == head2) done2 = 1;    // Line 18: c2 has looped back to start
}
while (!done1) {                  // Line 19: remaining terms of P1
    result = cpAddTerm(result, c1->coeff, c1->exp);
    c1 = c1->next;
    if (c1 == head1) done1 = 1;
}
while (!done2) {                  // Line 20: remaining terms of P2
    result = cpAddTerm(result, c2->coeff, c2->exp);
    c2 = c2->next;
    if (c2 == head2) done2 = 1;
}
return result;                    // Line 21
}

```

Line-by-line explanation: 1. Initialize empty result 2. c1 starts at head of first polynomial 3. c2 starts at head of second polynomial 4. Flags to detect when we've completed a full traversal of each circular list 5. Continue while neither list is fully processed 6. If current term of P1 has higher exponent, it belongs in result first 7. Add it to result circular list 8. Advance c1 9. If P2's current term has higher exponent 10-11. Add P2's term and advance c2 12. Equal exponents — add coefficients 13. Compute sum 14. Add to result only if sum is non-zero 15-16. Advance both pointers 17. If c1 has returned to head1, P1 is fully processed 18. Same for P2 19-20. Copy any remaining terms 21. Return result circular list

Dry Run — Add $P1 = 6x^3 + 4x^2 + 2x + 1$ and $P2 = 5x^3 + 3x^2 + 7$:

P1 circular: head1 -> [6|3] -> [4|2] -> [2|1] -> [1|0] -> (back to head1)

P2 circular: head2 -> [5|3] -> [3|2] -> [7|0] -> (back to head2)

done1=0, done2=0. c1=node(6,3), c2=node(5,3).

Iteration 1: c1->exp=3, c2->exp=3. Equal. sum=6+5=11. result=[11|3]. c1=node(4,2), c2=node(3,2).

c1 != head1, c2 != head2. done1=0, done2=0.

Iteration 2: c1->exp=2, c2->exp=2. Equal. sum=4+3=7. result=[11|3]->[7|2]. c1=node(2,1), c2=node(7,0).

done1=0, done2=0.

Iteration 3: c1->exp=1, c2->exp=0. 1>0 → Case A. result=[11|3]->[7|2]->[2|1]. c1=node(1,0).

c1 != head1. done1=0.

Iteration 4: c1->exp=0, c2->exp=0. Equal. sum=1+7=8. result=[11|3]->[7|2]->[2|1]->[8|0].

c1=head1. c2=head2. done1=1, done2=1.

Loop ends. No remaining terms.

Result: $11x^3 + 7x^2 + 2x + 8$ (circular)

Dry Run 2 — Add $P1 = 4x^4 + 3x + 2$ and $P2 = 2x^3 + x^2 + 5$:

P1: [4|4] -> [3|1] -> [2|0] -> (back to head1)

P2: [2|3] -> [1|2] -> [5|0] -> (back to head2)

Iter 1: $c1 \rightarrow \text{exp} = 4 > c2 \rightarrow \text{exp} = 3 \rightarrow$ Case A. $\text{result} = [4|4]$. $c1 = \text{node}(3,1)$. $\text{done1} = 0$.

Iter 2: $c1 \rightarrow \text{exp} = 1 < c2 \rightarrow \text{exp} = 3 \rightarrow$ Case B. $\text{result} = [4|4] \rightarrow [2|3]$. $c2 = \text{node}(1,2)$. $\text{done2} = 0$.

Iter 3: $c1 \rightarrow \text{exp} = 1 < c2 \rightarrow \text{exp} = 2 \rightarrow$ Case B. $\text{result} = [4|4] \rightarrow [2|3] \rightarrow [1|2]$. $c2 = \text{node}(5,0)$.
 $\text{done2} = 0$.

Iter 4: $c1 \rightarrow \text{exp} = 1 > c2 \rightarrow \text{exp} = 0 \rightarrow$ Case A. $\text{result} = [4|4] \rightarrow [2|3] \rightarrow [1|2] \rightarrow [3|1]$.
 $c1 = \text{node}(2,0)$. $\text{done1} = 0$.

Iter 5: $c1 \rightarrow \text{exp} = 0 == c2 \rightarrow \text{exp} = 0 \rightarrow$ Case C. $\text{sum} = 2 + 5 = 7$.
 $\text{result} = [4|4] \rightarrow [2|3] \rightarrow [1|2] \rightarrow [3|1] \rightarrow [7|0]$.
 $c1 = \text{head1}(\text{done1} = 1)$, $c2 = \text{head2}(\text{done2} = 1)$.

Result: $4x^4 + 2x^3 + x^2 + 3x + 7$

Polynomial Multiplication using Circular Singly Linked List

```
struct CPolyNode* cpMultiply(struct CPolyNode* head1, struct CPolyNode* head2) {
    struct CPolyNode* result = NULL;
    struct CPolyNode* c1 = head1;
    do {
        struct CPolyNode* c2 = head2;
        do {
            int newCoeff = c1->coeff * c2->coeff;
            int newExp = c1->exp + c2->exp;
            // Add to result or merge with existing same-exp term
            if (result == NULL) {
                result = cpAddTerm(result, newCoeff, newExp);
            } else {
                struct CPolyNode* t = result;
                int found = 0;
```

```

do {
    if (t->exp == newExp) { t->coeff += newCoeff; found = 1; break; }
    t = t->next;
} while (t != result);
if (!found) result = cpAddTerm(result, newCoeff, newExp);
}
c2 = c2->next;
} while (c2 != head2);
c1 = c1->next;
} while (c1 != head1);
return result;
}

```

Dry Run — Multiply $P1 = 2x^2 + 3x + 1$ by $P2 = x + 4$:

P1 circular: $[2|2] \rightarrow [3|1] \rightarrow [1|0] \rightarrow (\text{back})$

P2 circular: $[1|1] \rightarrow [4|0] \rightarrow (\text{back})$

$c1 = \text{node}(2,2)$:

$c2 = \text{node}(1,1)$: $2*1=2$, $2+1=3$. $\text{result}=[2|3]$.

$c2 = \text{node}(4,0)$: $2*4=8$, $2+0=2$. $\text{result}=[2|3] \rightarrow [8|2]$.

$c2 = \text{head2}$. Inner loop ends.

$c1 = \text{node}(3,1)$:

$c2 = \text{node}(1,1)$: $3*1=3$, $1+1=2$. $\text{exp}=2$ exists! $8+3=11$. $\text{result}=[2|3] \rightarrow [11|2]$.

$c2 = \text{node}(4,0)$: $3*4=12$, $1+0=1$. $\text{result}=[2|3] \rightarrow [11|2] \rightarrow [12|1]$.

$c2 = \text{head2}$. Inner loop ends.

$c1 = \text{node}(1,0)$:

c2=node(1,1): $1*1=1$, $0+1=1$. exp=1 exists! $12+1=13$. result=[2|3]->[11|2]->[13|1].

c2=node(4,0): $1*4=4$, $0+0=0$. result=[2|3]->[11|2]->[13|1]->[4|0].

c2=head2. Inner loop ends.

c1=head1. Outer loop ends.

Result: $2x^3 + 11x^2 + 13x + 4$

Verify: $(2x^2 + 3x + 1)(x + 4) = 2x^3 + 8x^2 + 3x^2 + 12x + x + 4 = 2x^3 + 11x^2 + 13x + 4$ ✓

Worked Problem 3 — Add Two Polynomials and Evaluate at $x = 2$

Given: $P1 = 3x^2 + 2x + 1$, $P2 = x^2 + 4x + 5$. Find $P3 = P1 + P2$, then evaluate $P3(2)$.

Step 1 — Build P1 and P2 as circular lists:

P1 circular: [3|2] -> [2|1] -> [1|0] -> (back to head1)

P2 circular: [1|2] -> [4|1] -> [5|0] -> (back to head2)

Step 2 — Add using cpAdd:

Iter 1: exp=2, exp=2. Equal. sum=3+1=4. result=[4|2]. Both advance.

Iter 2: exp=1, exp=1. Equal. sum=2+4=6. result=[4|2]->[6|1]. Both advance.

Iter 3: exp=0, exp=0. Equal. sum=1+5=6. result=[4|2]->[6|1]->[6|0]. Both return to heads.

$P3 = 4x^2 + 6x + 6$ (circular)

Step 3 — Evaluate P3 at $x=2$ by traversing circular list:

```
int cpEvaluate(struct CPolyNode* head, int x) {  
    if (!head) return 0;
```

```

int result = 0;
struct CPolyNode* curr = head;
do {
    int power = 1;
    for (int i = 0; i < curr->exp; i++) power *= x; // x^exp
    result += curr->coeff * power;
    curr = curr->next;
} while (curr != head);
return result;
}

```

Evaluation of $4x^2 + 6x + 6$ at $x=2$:

$\text{curr} = \text{node}(4,2)$: $\text{power} = 2^2 = 4$. $\text{result} += 4*4 = 16$. $\text{result} = 16$.

$\text{curr} = \text{node}(6,1)$: $\text{power} = 2^1 = 2$. $\text{result} += 6*2 = 12$. $\text{result} = 28$.

$\text{curr} = \text{node}(6,0)$: $\text{power} = 2^0 = 1$. $\text{result} += 6*1 = 6$. $\text{result} = 34$.

$\text{curr} = \text{head}$. Stop.

$P_3(2) = 34$.

Verify: $4(4) + 6(2) + 6 = 16 + 12 + 6 = 34$ ✓

 **Exam Tip:** The evaluation loop over a circular polynomial list is a high-yield exam sub-question. The pattern is: use do-while, compute x^{exp} by a nested loop (or use `pow()`), accumulate $\text{coeff} * x^{\text{exp}}$, stop when back at head. State the do-while termination condition explicitly.

Time and Space Complexity – Polynomial Circular Linked List

Operation	Time	Space
Creation (n terms)	$O(n)$	$O(n)$
Display (n terms)	$O(n)$	$O(1)$
Addition (m, n terms)	$O(m + n)$	$O(m + n)$
Multiplication (m, n terms)	$O(m * n * (m+n))$	$O(m * n)$
Evaluation (n terms)	$O(n * \text{max_exp})$	$O(1)$

1.9 Polynomial Operations using Queue

Priority: Tier 1 – Maximum Depth

Concept

A queue-based approach to polynomial operations treats each polynomial as a sequence of (coefficient, exponent) pairs enqueued in decreasing order of exponent. The FIFO property of a queue means the front always holds the term with the highest remaining exponent, enabling a clean merge-based addition similar to merge sort.

Polynomial $P1 = 6x^3 + 4x^2 + 2x + 1$ stored as a queue:

Front $\rightarrow (6,3) \rightarrow (4,2) \rightarrow (2,1) \rightarrow (1,0) \leftarrow$ Rear

Polynomial $P2 = 5x^3 + 3x^2 + 7$ stored as a queue:

Front $\rightarrow (5,3) \rightarrow (3,2) \rightarrow (7,0) \leftarrow$ Rear

Data Structures

```
struct PolyTerm {
    int coeff;
    int exp;
};

#define MAX 100

struct Queue {
    struct PolyTerm terms[MAX];
    int front, rear;
};

void initQueue(struct Queue *q) { q->front = q->rear = -1; }

int isEmpty(struct Queue *q) { return q->front == -1; }

void enqueue(struct Queue *q, int coeff, int exp) {
    if (q->rear == MAX - 1) { printf("Queue overflow\n"); return; }
    if (q->front == -1) q->front = 0;
    q->rear++;
    q->terms[q->rear].coeff = coeff;
    q->terms[q->rear].exp = exp;
}
```

```

struct PolyTerm dequeue(struct Queue *q) {
    struct PolyTerm t = q->terms[q->front];
    if (q->front == q->rear) { q->front = q->rear = -1; }
    else q->front++;
    return t;
}

```

```

struct PolyTerm peek(struct Queue *q) { return q->terms[q->front]; }

```

ASCII Diagram – Two Polynomial Queues Before Addition

Q1 ($P1 = 6x^3 + 4x^2 + 2x + 1$):

front=0, rear=3

Index: [0]	[1]	[2]	[3]
(6,3)	(4,2)	(2,1)	(1,0)
^		^	
front		rear	

Q2 ($P2 = 5x^3 + 3x^2 + 7$):

front=0, rear=2

Index: [0]	[1]	[2]
(5,3)	(3,2)	(7,0)
^	^	
front	rear	

Result Queue (initially empty):

front=-1, rear=-1

Polynomial Addition using Queue

Algorithm: 1. Initialize Q1, Q2 with the respective polynomial terms (decreasing exponent order) 2. Initialize empty result queue QR 3. While neither Q1 nor Q2 is empty: - Peek at front of each - Case A: $Q1.front.exp > Q2.front.exp$: dequeue from Q1, enqueue to QR - Case B: $Q1.front.exp < Q2.front.exp$: dequeue from Q2, enqueue to QR - Case C: equal exponents: dequeue both, add coefficients, enqueue sum to QR (if $\neq 0$) 4. Drain remaining non-empty queue into QR 5. Display or return QR

```
void addPolyQueue(struct Queue *q1, struct Queue *q2, struct Queue *qr) {
```

```
    initQueue(qr);                                // Line 1

    while (!isEmpty(q1) && !isEmpty(q2)) {          // Line 2
        struct PolyTerm t1 = peek(q1);             // Line 3
        struct PolyTerm t2 = peek(q2);             // Line 4

        if (t1.exp > t2.exp) {                      // Line 5: Case A
            struct PolyTerm t = dequeue(q1);        // Line 6
            enqueue(qr, t.coeff, t.exp);            // Line 7
        } else if (t1.exp < t2.exp) {               // Line 8: Case B
            struct PolyTerm t = dequeue(q2);        // Line 9
            enqueue(qr, t.coeff, t.exp);            // Line 10
        } else {                                    // Line 11: Case C
            dequeue(q1);                             // Line 12
            dequeue(q2);                             // Line 13
            int sum = t1.coeff + t2.coeff;          // Line 14
            if (sum != 0) enqueue(qr, sum, t1.exp); // Line 15
        }
    }
}
```

```

while (!isEmpty(q1)) {                                // Line 16
    struct PolyTerm t = dequeue(q1);
    enqueue(qr, t.coeff, t.exp);
}
while (!isEmpty(q2)) {                                // Line 17
    struct PolyTerm t = dequeue(q2);
    enqueue(qr, t.coeff, t.exp);
}
}

```

Line-by-line explanation: 1. Initialize result queue (front=rear=-1) 2. Process while both queues have terms 3. Peek at Q1's front term without removing 4. Peek at Q2's front term without removing 5. Q1's front has higher exponent 6. Dequeue from Q1 7. Add to result 8. Q2's front has higher exponent 9. Dequeue from Q2 10. Add to result 11. Equal exponents — merge 12. Remove from Q1 13. Remove from Q2 14. Add coefficients 15. Only enqueue if sum is non-zero (zero terms are dropped) 16. Drain remaining Q1 terms 17. Drain remaining Q2 terms

Dry Run — Add $Q1 = [6x^3 + 4x^2 + 2x + 1]$ and $Q2 = [5x^3 + 3x^2 + 7]$:

Initial:

Q1: (6,3),(4,2),(2,1),(1,0)

Q2: (5,3),(3,2),(7,0)

QR: empty

Step 1: $t1=(6,3)$, $t2=(5,3)$. exp equal. $sum=6+5=11$. QR: (11,3). Q1:(4,2),(2,1),(1,0). Q2:(3,2),(7,0).

Step 2: $t1=(4,2)$, $t2=(3,2)$. exp equal. $sum=4+3=7$. QR: (11,3),(7,2). Q1:(2,1),(1,0). Q2:(7,0).

Step 3: $t1=(2,1)$, $t2=(7,0)$. $1>0 \rightarrow$ Case A. QR: $(11,3),(7,2),(2,1)$. Q1: $(1,0)$. Q2: $(7,0)$.

Step 4: $t1=(1,0)$, $t2=(7,0)$. exp equal. $sum=1+7=8$. QR: $(11,3),(7,2),(2,1),(8,0)$. Both queues empty.

Result QR: $11x^3 + 7x^2 + 2x + 8$

Dry Run 2 — Add Q1 = $[3x^4 + 2x^2]$ and Q2 = $[5x^3 + 4x^2 + 1]$:

Q1: $(3,4),(2,2)$

Q2: $(5,3),(4,2),(1,0)$

Step 1: $t1=(3,4)$, $t2=(5,3)$. $4>3 \rightarrow$ Case A. QR: $(3,4)$. Q1: $(2,2)$.

Step 2: $t1=(2,2)$, $t2=(5,3)$. $2<3 \rightarrow$ Case B. QR: $(3,4),(5,3)$. Q2: $(4,2),(1,0)$.

Step 3: $t1=(2,2)$, $t2=(4,2)$. equal. $sum=2+4=6$. QR: $(3,4),(5,3),(6,2)$. Q1: empty. Q2: $(1,0)$.

Drain Q2: enqueue $(1,0)$. QR: $(3,4),(5,3),(6,2),(1,0)$.

Result: $3x^4 + 5x^3 + 6x^2 + 1$

Polynomial Multiplication using Queue

```
void multiplyPolyQueue(struct Queue *q1, struct Queue *q2, struct Queue *qr) {
    initQueue(qr);

    // We need to iterate q1 in full for each element of q2.

    // Save q1 into an array first (since dequeue is destructive).
    struct PolyTerm arr1[MAX]; int n1 = 0;

    struct Queue q1copy = *q1;
    while (!isEmpty(&q1copy)) arr1[n1++] = dequeue(&q1copy);

    struct PolyTerm arr2[MAX]; int n2 = 0;
    struct Queue q2copy = *q2;
```

```

while (!isEmpty(&q2copy)) arr2[n2++] = dequeue(&q2copy);

// Multiply all pairs
// Store results temporarily in a buffer, then sort and merge
struct PolyTerm temp[MAX * MAX]; int nt = 0;
for (int i = 0; i < n1; i++) {
    for (int j = 0; j < n2; j++) {
        temp[nt].coeff = arr1[i].coeff * arr2[j].coeff;
        temp[nt].exp = arr1[i].exp + arr2[j].exp;
        nt++;
    }
}

// Sort by decreasing exponent (bubble sort)
for (int i = 0; i < nt - 1; i++)
    for (int j = 0; j < nt - i - 1; j++)
        if (temp[j].exp < temp[j+1].exp) {
            struct PolyTerm t = temp[j]; temp[j] = temp[j+1]; temp[j+1] = t;
        }

// Merge same-exponent terms into qr
for (int i = 0; i < nt; ) {
    int coeff = temp[i].coeff, exp = temp[i].exp;
    int j = i + 1;
    while (j < nt && temp[j].exp == exp) { coeff += temp[j].coeff; j++; }
    if (coeff != 0) enqueue(qr, coeff, exp);
    i = j;
}
}

```

Dry Run — Multiply $Q1 = [2x^2 + 3x]$ by $Q2 = [x + 2]$:

arr1: (2,2),(3,1)

arr2: (1,1),(2,0)

All pairs:

$(2,2) * (1,1) = (2,3)$

$(2,2) * (2,0) = (4,2)$

$(3,1) * (1,1) = (3,2)$

$(3,1) * (2,0) = (6,1)$

temp: (2,3),(4,2),(3,2),(6,1)

After sort by decreasing exp: (2,3),(4,2),(3,2),(6,1)

Merge same exp:

exp=3: coeff=2. Enqueue (2,3).

exp=2: 4+3=7. Enqueue (7,2).

exp=1: 6. Enqueue (6,1).

Result QR: $2x^3 + 7x^2 + 6x$

Verify: $(2x^2 + 3x)(x + 2) = 2x^3 + 4x^2 + 3x^2 + 6x = 2x^3 + 7x^2 + 6x$ ✓

Worked Problem 3 — Add Two Queue Polynomials and Display in Descending Order

Given: $Q1 = 5x^4 + 2x^2 + 3$, $Q2 = 4x^4 + 3x^3 + x^2 + 7x$. Find: $Q1 + Q2$, displayed in descending order of exponent.

Q1: (5,4),(2,2),(3,0)

Q2: (4,4),(3,3),(1,2),(7,1)

Step 1: t1=(5,4), t2=(4,4). equal. sum=9. QR:(9,4). Q1:(2,2),(3,0). Q2:(3,3),(1,2),(7,1).

Step 2: t1=(2,2), t2=(3,3). $2 < 3 \rightarrow$ Case B. QR:(9,4),(3,3). Q2:(1,2),(7,1).

Step 3: t1=(2,2), t2=(1,2). equal. sum=3. QR:(9,4),(3,3),(3,2). Q1:(3,0). Q2:(7,1).

Step 4: t1=(3,0), t2=(7,1). $0 < 1 \rightarrow$ Case B. QR:(9,4),(3,3),(3,2),(7,1). Q2:empty.

Drain Q1: enqueue (3,0). QR:(9,4),(3,3),(3,2),(7,1),(3,0).

Result (already in descending order): $9x^4 + 3x^3 + 3x^2 + 7x + 3$

Display: dequeue each term and print:

$9x^4 + 3x^3 + 3x^2 + 7x + 3$

Edge Cases for All Polynomial Representations

Case	Behavior
Empty polynomial (NULL/empty queue)	Return other polynomial unchanged
Single-term polynomial	Works — loop runs once
No overlapping exponents	All terms Case A or B; result is union of both
All exponents overlap	All terms Case C; result has same term count as inputs

Case	Behavior
Coefficients cancel to zero	Omit the term from result (sum == 0 check)

Comparison Table — Polynomial Representations

Feature	Singly Linked List	Circular Linked List	Queue
Add Time	$O(m+n)$	$O(m+n)$	$O(m+n)$
Multiply Time	$O(mn(m+n))$	$O(mn(m+n))$	$O(mn \log(m \cdot n))$
Traversal	While($\neq \text{NULL}$)	do-while($\neq \text{head}$)	While(!isEmpty)
NULL in list?	Yes (at end)	No	N/A
Restart inner loop	Requires head pointer	Natural: circular	Requires array copy
Evaluation loop	While($\neq \text{NULL}$)	do-while($\neq \text{head}$)	Dequeue until empty
Space	$O(n)$	$O(n)$	$O(n)$

📌 Exam Tip: The core logic (3 exponent cases, zero-sum elimination, drain remaining) is IDENTICAL across all three representations. The only difference is the loop termination condition. Mastering the singly-linked-list version means you know the queue version — just swap `while(temp!=NULL)` for `while(!isEmpty(q))` and `temp=temp->next` for `t=dequeue(q)`.

Notes are incomplete. Type continue to resume from Module 3: Stacks and Queues.

MODULE 3: STACKS AND QUEUES

2.1 Introduction to Stacks

Priority: Tier 3

A stack is a linear data structure that follows the LIFO (Last In, First Out) principle. The last element inserted is the first element removed. Think of a stack of plates: you add to the top and remove from the top.

Stack State (top = index 2):

Index: [0] [1] [2]

Data: [10][20][30]

^

top

Push(40): top becomes 3, stack = [10][20][30][40]

Pop(): removes 40, top becomes 2 again

Real-world analogies: stack of books, browser back button, undo operation, function call stack.

Key terms: - Top: index/pointer indicating the topmost occupied slot - Overflow: attempt to push when stack is full - Underflow: attempt to pop when stack is empty

2.2 Array-Based Stack

Priority: Tier 1

Implementation

```
#include <stdio.h>

#define MAX 100

int stack[MAX];
int top = -1; // -1 means empty

int isFull() { return top == MAX - 1; }
int isEmpty() { return top == -1; }

void push(int val) {
    if (isFull()) { printf("Stack Overflow\n"); return; } // Line 1
    stack[++top] = val; // Line 2
    printf("Pushed: %d\n", val); // Line 3
}

int pop() {
    if (isEmpty()) { printf("Stack Underflow\n"); return -1; } // Line 4
```

```
    return stack[top--];                // Line 5
}

void display() {
    if (isEmpty()) { printf("Stack empty\n"); return; }
    for (int i = top; i >= 0; i--)
        printf("%d ", stack[i]);
    printf("\n");
}
```

Line-by-line (push): 1. Check overflow: if top is at MAX-1, no more space 2. Pre-increment top to next free slot, store value there 3. Confirm push

Line-by-line (pop): 4. Check underflow: nothing to remove 5. Return current top element, then decrement top (post-decrement)

Dry Run — Push 10, 20, 30, then Pop twice:

Initial: top=-1. Stack: []

Push 10: top=0. Stack: [10]

Push 20: top=1. Stack: [10,20]

Push 30: top=2. Stack: [10,20,30]

Pop: returns 30, top=1. Stack: [10,20]

Pop: returns 20, top=0. Stack: [10]

2.3 Linked List-Based Stack

Priority: Tier 2

```

struct StackNode {
    int data;
    struct StackNode *next;
};

struct StackNode* top_ptr = NULL;

void IIPush(int val) {
    struct StackNode* newNode = (struct StackNode*)malloc(sizeof(struct
StackNode));
    newNode->data = val;
    newNode->next = top_ptr; // new node points to old top
    top_ptr = newNode;      // new node becomes new top
}

int IIPop() {
    if (top_ptr == NULL) { printf("Underflow\n"); return -1; }
    int val = top_ptr->data;
    struct StackNode* temp = top_ptr;
    top_ptr = top_ptr->next;
    free(temp);
    return val;
}

```

2.4 Infix to Postfix Conversion

Priority: Tier 1 — Maximum Depth

Concept

Infix notation: operators between operands: $A + B * C$ Postfix notation: operators after operands: $A B C * +$

Postfix is preferred for machine evaluation because it requires no parentheses and no precedence rules — expressions are evaluated strictly left to right.

Operator Precedence Table

Operator	Precedence	Associativity
(	0 (lowest when in stack)	—
+, -	1	Left to right
*, /, %	2	Left to right
^	3 (highest)	Right to left

Algorithm

- Initialize empty stack S. Initialize empty string postfix P.
- Push (onto S. Append) to end of infix expression.
- Scan the infix expression left to right:
 - **Operand** (letter or digit): append to P
 - **Left parenthesis** (: push onto S
 - **Operator op**:

- While S is not empty AND precedence(S.top) >= precedence(op)
AND S.top != (:
 - Pop from S and append to P
- Push op onto S
- **Right parenthesis):**
 - While S.top != (: pop and append to P
 - Pop and discard the (
- P is the postfix expression.

 **Exam Tip:** The most common exam error is forgetting to pop when a right parenthesis is encountered (the (must be discarded, not added to P). Also, ^ uses right-to-left associativity: pop only operators with strictly HIGHER precedence, not equal.

C Implementation

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char stack[100];
int top = -1;

void push(char c) { stack[++top] = c; }
char pop()      { return stack[top--]; }
char peek()     { return stack[top]; }
int isEmpty()   { return top == -1; }
```

```

int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    if (op == '^') return 3;
    return 0;
}

void infixToPostfix(char *infix, char *postfix) {
    int j = 0;
    push('('); // Line 1
    int len = strlen(infix);
    infix[len] = ')'; infix[len+1] = '\0'; // Line 2
    for (int i = 0; infix[i] != '\0'; i++) { // Line 3
        char ch = infix[i];
        if (isalnum(ch)) { // Line 4
            postfix[j++] = ch; // Line 5
        } else if (ch == '(') { // Line 6
            push(ch); // Line 7
        } else if (ch == ')') { // Line 8
            while (peek() != '(') postfix[j++] = pop(); // Line 9
            pop(); // discard '(' // Line 10
        } else { // Line 11: operator
            while (!isEmpty() && peek() != '(' &&
                precedence(peek()) >= precedence(ch)) // Line 12
                postfix[j++] = pop(); // Line 13
            push(ch); // Line 14
        }
    }
}

```

```
    postfix[j] = '\0';                // Line 15
}
```

Line-by-line explanation: 1. Push sentinel (to simplify logic 2. Append) to infix so the sentinel (gets popped at the end 3. Scan each character 4. Check if character is alphanumeric (operand) 5. Append operand directly to postfix 6. Left parenthesis encountered 7. Push it — increases precedence scope 8. Right parenthesis encountered 9. Pop and output everything until matching (10. Discard the (itself 11. Any other character is an operator 12. Pop while top has equal-or-higher precedence (respects left associativity) 13. Add popped operators to postfix 14. Push current operator 15. Null-terminate postfix string

Dry Run — Convert $A + B * C - D$ to postfix:

Infix: $A+B*C-D$) (after appending))

Stack initially: []

Scan A: operand. Postfix: A

Scan +: op, prec=1. peek=(, prec=0. $1 \geq 0$? YES but peek=(, skip. Push +.

Stack: [(,+] Postfix: A

Scan B: operand. Postfix: AB

Scan *: op, prec=2. peek=+, prec=1. $2 \geq 1$? YES, but precedence(*)=2 $\geq$ precedence(+)=1 means we should

pop... wait, $2 \geq 1$ so we DON'T pop yet — actually we DO because precedence(*) = $2 \geq$ precedence(+) = 1.

Wait, let me redo: we pop while precedence(peek) $\geq$ precedence(current).

peek=+, prec(+)=1. prec(*)=2. Is prec(+)=1 >= prec(*)=2? NO. Don't pop. Push *.

Stack: [(, *] Postfix: AB

Scan C: operand. Postfix: ABC

Scan -: op, prec=1.

peek=*, prec=2. 2 >= 1? YES. Pop *. Postfix: ABC*

peek=+, prec=1. 1 >= 1? YES. Pop +. Postfix: ABC*+

peek=(, skip. Push -.

Stack: [(, -] Postfix: ABC*+

Scan D: operand. Postfix: ABC*+D

Scan):

pop until (: pop -. Postfix: ABC*+D-

pop and discard (.

Final Postfix: ABC*+D-

Verify: $A + BC - D = A + (BC) - D$ Postfix: A B C * + D - Read: push A, push B, push C, pop C and B $\rightarrow BC$, push result, pop A and (BC) $\rightarrow A+(BC)$, push D, pop D and (A+BC) $\rightarrow$ final. Result: $A + B * C - D$ ✓

Dry Run 2 — Convert $(A + B) * (C - D)$ to postfix:

Infix: $(A+B)*(C-D)$

Stack: [(]

(: push (. Stack: [(, (]

A : Postfix: A

+ : prec=1. peek=(, skip. Push +. Stack: [(, +]

B : Postfix: AB

) : pop until (: pop +. Postfix: AB+. Discard (. Stack: [()

* : prec=2. peek=(, skip. Push *. Stack: [(, *,]

(: push (. Stack: [(, *, (]

C : Postfix: AB+C

- : prec=1. peek=(, skip. Push -. Stack: [(, *, (-]

D : Postfix: AB+CD

) : pop until (: pop -. Postfix: AB+CD-. Discard (. Stack: [(, *,]

) : pop until (: pop *. Postfix: AB+CD-*. Discard (. Stack: [()]

Final Postfix: AB+CD-*

= (A+B)*(C-D) ✓

 Exam Tip: For a 5-mark infix-to-postfix question, always show a table with columns: Character | Stack State | Postfix So Far. This organized format demonstrates full understanding and earns all marks.

Tabular Dry Run Format (Exam-Ready)

For $A + B * C - D$:

Character	Action	Stack	Postfix
A	output	(	A
+	push	(+	A

Character	Action	Stack	Postfix
B	output	(+	AB
*	prec(*) > prec(+), push	(+ *	AB
C	output	(+ *	ABC
-	pop * (2>=1), pop + (1>=1), push -	(-	ABC*+
D	output	(-	ABC*+D
)	pop -	(	ABC*+D-

Final: ABC*+D-

5-Mark Answer Blueprint — Infix to Postfix

- **Precedence table:** +, - = 1; *, / = 2; ^ = 3
- **Algorithm** (7 numbered steps from lecture slides)
- **Tabular dry run** on given expression with columns: Char | Stack | Postfix
- **Result:** clearly state the final postfix expression
- **Complexity:** O(n) time, O(n) space

2.5 Postfix Evaluation

Priority: Tier 1

Algorithm

- Scan postfix expression left to right.
- If **operand**: push onto stack.
- If **operator**: pop top two operands ($b = \text{pop}$, $a = \text{pop}$), compute $a \text{ OP } b$, push result.
- Final answer is the single remaining element on the stack.

Note: Pop order matters. First pop is the right operand; second pop is the left operand. For $A B -$, we get $b=B$, $a=A$, result = $A - B$ (not $B - A$).

C Implementation

```
int evaluatePostfix(char *postfix) {
    int stack[100], top = -1;
    for (int i = 0; postfix[i] != '\0'; i++) {    // Line 1
        char ch = postfix[i];
        if (isdigit(ch)) {                        // Line 2
            stack[++top] = ch - '0';              // Line 3
        } else {                                  // Line 4: operator
            int b = stack[top--];                  // Line 5
            int a = stack[top--];                  // Line 6
            if (ch == '+') stack[++top] = a + b; // Line 7
            else if (ch == '-') stack[++top] = a - b; // Line 8
            else if (ch == '*') stack[++top] = a * b; // Line 9
            else if (ch == '/') stack[++top] = a / b; // Line 10
        }
    }
    return stack[top];                            // Line 11
}
```

Line-by-line: 1. Scan each character 2. Single-digit operand check 3. Convert char digit to integer, push 4. All other characters are operators 5. Pop right operand (b is the second operand) 6. Pop left operand (a is the first operand) 7-10. Compute and push result 11. Return final answer

Dry Run — Evaluate $23*4+$:

Stack: []

2: push 2. Stack: [2]

3: push 3. Stack: [2,3]

*: $b=\text{pop}=3$, $a=\text{pop}=2$. $2*3=6$. push 6. Stack: [6]

4: push 4. Stack: [6,4]

+: $b=\text{pop}=4$, $a=\text{pop}=6$. $6+4=10$. push 10. Stack: [10]

Result: 10

Verify: $(2*3)+4 = 6+4 = 10$ ✓

Dry Run 2 — Evaluate $532+*$:

Stack: []

5: push 5. [5]

3: push 3. [5,3]

2: push 2. [5,3,2]

+: $b=2$, $a=3$. $3+2=5$. push 5. [5,5]

*: $b=5$, $a=5$. $5*5=25$. push 25. [25]

Result: 25

Verify: $5*(3+2) = 5*5 = 25$ ✓

📌 Exam Tip: Always show stack state after each token. The pop order (second pop = left operand) is the most commonly tested error. Show it explicitly: “b = pop (right), a = pop (left), compute a OP b”.

Time Complexity – Stack Operations

Operation	Time	Space
Infix to Postfix	$O(n)$	$O(n)$
Postfix Evaluation	$O(n)$	$O(n)$
Push/Pop	$O(1)$	$O(1)$

2.6 Queues

Priority: Tier 3

A queue is a linear data structure following FIFO (First In, First Out). Elements are inserted at the rear and removed from the front.

Queue: [10][20][30][40]

 ^ ^
front rear

Enqueue(50): [10][20][30][40][50] rear advances

Dequeue(): [20][30][40][50] front advances

Array-Based Queue

```
#define MAX 100

int queue[MAX], front = -1, rear = -1;

int isQFull() { return rear == MAX - 1; }
int isQEmpty() { return front == -1 || front > rear; }

void enqueue(int val) {
    if (isQFull()) { printf("Queue Overflow\n"); return; }
    if (front == -1) front = 0;
    queue[++rear] = val;
}

int dequeue() {
    if (isQEmpty()) { printf("Queue Underflow\n"); return -1; }
    int val = queue[front++];
    if (front > rear) front = rear = -1; // reset when empty
    return val;
}
```

Linked List-Based Queue

```
struct QNode {
    int data;
    struct QNode *next;
};

struct QNode *front_ptr = NULL, *rear_ptr = NULL;
```

```
void llEnqueue(int val) {
    struct QNode* newNode = (struct QNode*)malloc(sizeof(struct QNode));
    newNode->data = val; newNode->next = NULL;
    if (rear_ptr == NULL) { front_ptr = rear_ptr = newNode; return; }
    rear_ptr->next = newNode;
    rear_ptr = newNode;
}

int llDequeue() {
    if (front_ptr == NULL) { printf("Underflow\n"); return -1; }
    int val = front_ptr->data;
    struct QNode* temp = front_ptr;
    front_ptr = front_ptr->next;
    if (front_ptr == NULL) rear_ptr = NULL;
    free(temp);
    return val;
}
```

2.7 Circular Queue

Priority: Tier 1 — Maximum Depth

Motivation

In a linear array queue, after several enqueue/dequeue operations, front and rear keep advancing rightward. Even when the queue logically has space at the beginning

(positions 0 to front-1 have been freed), those slots cannot be reused. This wastes memory.

Linear Queue Problem:

After 3 enqueues and 2 dequeues:

Array: [][][30][40][50]

 ^ ^
 front rear

Positions 0 and 1 are free but UNREACHABLE.

Even after enqueueing MAX-3 more elements, only 3 more fit.

Circular Queue Solution: Treat the array as circular. After the last index (MAX-1), wrap around to index 0.

Circular Queue (MAX=5):

Indices: [0][1][2][3][4]

 ^
 rear can wrap here after index 4

ENQUEUE formula: $\text{rear} = (\text{rear} + 1) \% \text{MAX}$

DEQUEUE formula: $\text{front} = (\text{front} + 1) \% \text{MAX}$

Implementation

```
#define MAX 5  
  
int cq[MAX];  
  
int front = -1, rear = -1;  
  
int cqFull() { return (rear + 1) % MAX == front; }  
int cqEmpty() { return front == -1; }
```

```
void cqEnqueue(int val) {
    if (cqFull()) { printf("Circular Queue Overflow\n"); return; } // Line 1
    if (front == -1) front = 0; // Line 2
    rear = (rear + 1) % MAX; // Line 3
    cq[rear] = val; // Line 4
    printf("Enqueued: %d\n", val); // Line 5
}

int cqDequeue() {
    if (cqEmpty()) { printf("Circular Queue Underflow\n"); return -1; } // Line 6
    int val = cq[front]; // Line 7
    if (front == rear) { front = rear = -1; } // Line 8
    else front = (front + 1) % MAX; // Line 9
    return val; // Line 10
}

void cqDisplay() {
    if (cqEmpty()) { printf("Queue empty\n"); return; }
    int i = front;
    do {
        printf("%d ", cq[i]);
        i = (i + 1) % MAX;
    } while (i != (rear + 1) % MAX);
    printf("\n");
}
```

Line-by-line explanation: 1. Full condition: one slot is always reserved to distinguish full from empty. Full when the next rear position equals front. 2. Initialize front when first element is inserted 3. Advance rear with modular arithmetic — wraps from MAX-1 to 0 4. Store value at new rear 5. Confirm 6. Underflow check 7. Save value to return 8. If front == rear, this was the last element; reset to -1 9. Advance front with modular arithmetic 10. Return the dequeued value

📌 Exam Tip: The full condition $(\text{rear} + 1) \% \text{MAX} == \text{front}$ wastes one slot.

An alternative is to keep a count variable. Both are acceptable in exams; state which approach you're using.

Dry Run — Enqueue 10, 20, 30, 40, then Dequeue twice, then Enqueue 50, 60 (MAX=5):

MAX = 5. Indices 0-4.

Initial: front=-1, rear=-1.

Enqueue 10: front=0, rear=0. cq=[10,_,_,_,_]

Enqueue 20: rear=1. cq=[10,20,_,_,_]

Enqueue 30: rear=2. cq=[10,20,30,_,_]

Enqueue 40: rear=3. cq=[10,20,30,40,_]

Dequeue: returns 10. front=(0+1)%5=1. cq=[_,20,30,40,_]. front=1, rear=3.

Dequeue: returns 20. front=(1+1)%5=2. cq=[_,_,30,40,_]. front=2, rear=3.

Enqueue 50: rear=(3+1)%5=4. cq=[_,_,30,40,50]. front=2, rear=4.

Enqueue 60: rear=(4+1)%5=0. cq=[60,_,30,40,50]. front=2, rear=0.

Full check: $(0+1)\%5=1 == \text{front}=2$? NO. Not full.

Enqueue 70: $\text{rear} = (0+1)\%5 = 1$. $\text{cq} = [60, 70, 30, 40, 50]$. $\text{front} = 2$, $\text{rear} = 1$.

Full check: $(1+1)\%5 = 2 == \text{front} = 2$? YES. Full now.

Dequeue: returns $\text{cq}[2] = 30$. $\text{front} = (2+1)\%5 = 3$.

Display from $\text{front} = 3$:

$i = 3$: 40, $i = 4$: 50, $i = 0$: 60, $i = 1$: 70. Stop when $i = (\text{rear}+1)\% \text{MAX} = (1+1)\%5 = 2$.

Output: 40 50 60 70

Visual State After All Operations:

Index: [0] [1] [2] [3] [4]

Data: [60][70][][40][50]

^ ^

rear front

(Queue contains: 40, 50, 60, 70)

 **Exam Tip:** The wrap-around diagram is what examiners look for. Always draw the array with indices and show front/rear positions explicitly at each step.

5-Mark Answer Blueprint – Circular Queue

- **Motivation** (1 sentence): explain memory waste in linear queue
- **Full/Empty conditions:** $(\text{rear}+1)\% \text{MAX} == \text{front}$ for full; $\text{front} == -1$ for empty
- **Modular arithmetic formula:** $\text{rear} = (\text{rear}+1)\% \text{MAX}$ for enqueue; $\text{front} = (\text{front}+1)\% \text{MAX}$ for dequeue
- **C code:** 15-20 lines covering enqueue and dequeue

- **Dry run:** show at least 4 enqueues, 2 dequeues, and a wrap-around enqueue with array states
-

2.8 Priority Queue

Priority: Tier 2

A priority queue is a queue in which each element has an associated priority. Elements with higher priority are dequeued before elements with lower priority, regardless of their insertion order. Among equal priorities, FIFO applies.

Rules

- Higher priority element is served before lower priority.
- Equal priority: served in order of arrival (FIFO).

Implementation (Array-Based, Ascending Priority = Higher Value = Higher Priority)

```
struct PQNode {
    int data;
    int priority;
};

struct PQNode pq[100];
int pqSize = 0;

void pqInsert(int data, int priority) {
    pq[pqSize].data = data;
    pq[pqSize].priority = priority;
```

```

    pqSize++;
}

int pqDelete() {
    if (pqSize == 0) { printf("Priority Queue empty\n"); return -1; }
    int maxIdx = 0;
    for (int i = 1; i < pqSize; i++)
        if (pq[i].priority > pq[maxIdx].priority)
            maxIdx = i;
    int val = pq[maxIdx].data;
    // Shift remaining elements
    for (int i = maxIdx; i < pqSize - 1; i++)
        pq[i] = pq[i + 1];
    pqSize--;
    return val;
}

```

Dry Run — Insert (A,3), (B,1), (C,2), then Delete:

Insert (A,3): pq = [(A,3)]

Insert (B,1): pq = [(A,3),(B,1)]

Insert (C,2): pq = [(A,3),(B,1),(C,2)]

Delete: scan for max priority. maxIdx=0 (A,3 has priority 3, highest).

Returns A. pq = [(B,1),(C,2)].

Delete: maxIdx=1 (C,2 has higher priority than B,1).

Returns C. pq = [(B,1)].

Delete: Returns B.

MODULE 4: SORTING, SEARCHING, AND HASHING

3.1 Linear Search

Priority: Tier 2

Concept

Linear search (sequential search) scans each element from the start until the target is found or the list is exhausted.

Iterative Implementation

```
int linearSearch(int arr[], int n, int key) {  
    for (int i = 0; i < n; i++) {    // Line 1  
        if (arr[i] == key) return i; // Line 2  
    }  
    return -1;                      // Line 3  
}
```

Dry Run — Search for 30 in [10, 25, 30, 45, 60, 5]:

```
i=0: arr[0]=10 == 30? NO.  
i=1: arr[1]=25 == 30? NO.  
i=2: arr[2]=30 == 30? YES. Return 2.
```

Recursive Implementation

```
int linearSearchRec(int arr[], int n, int key, int i) {  
    if (i == n) return -1;           // base case: not found  
    if (arr[i] == key) return i;     // base case: found  
    return linearSearchRec(arr, n, key, i + 1); // recurse  
}
```

Complexity

Case	Comparisons	Time
Best	1 (first element)	$O(1)$
Average	$n/2$	$O(n)$
Worst	n (last or absent)	$O(n)$
Space	—	$O(1)$ iterative, $O(n)$ recursive

3.2 Binary Search

Priority: Tier 1

Concept

Binary search works only on sorted arrays. It repeatedly halves the search space by comparing the target with the middle element.

Array (sorted): [5, 10, 20, 30, 45, 60]

Indices: 0 1 2 3 4 5

Iterative Implementation

```
int binarySearchIter(int arr[], int n, int key) {  
    int low = 0, high = n - 1;           // Line 1  
    while (low <= high) {                 // Line 2  
        int mid = low + (high - low) / 2; // Line 3  
        if (arr[mid] == key) return mid;  // Line 4  
        else if (arr[mid] < key) low = mid + 1; // Line 5  
        else high = mid - 1;             // Line 6  
    }  
    return -1;                            // Line 7  
}
```

Line-by-line: 1. Initialize search boundaries 2. Continue while valid range remains 3. Compute mid (using subtraction to avoid integer overflow) 4. Found! Return index 5. Target is in right half — eliminate left half 6. Target is in left half — eliminate right half 7. Target not found

Dry Run — Search for 30 in [5, 10, 20, 30, 45, 60]:

low=0, high=5.

Step 1: $mid = 0 + (5-0)/2 = 2$. $arr[2]=20$. $20 < 30 \rightarrow low = 3$.

Step 2: low=3, high=5. $mid = 3 + (5-3)/2 = 4$. $arr[4]=45$. $45 > 30 \rightarrow high = 3$.

Step 3: low=3, high=3. $mid = 3$. $arr[3]=30 == 30$. Return 3. ✓

Dry Run 2 – Search for 15 in [5, 10, 20, 30, 45, 60]:

low=0, high=5.

Step 1: mid=2. arr[2]=20 > 15 → high=1.

Step 2: low=0, high=1. mid=0. arr[0]=5 < 15 → low=1.

Step 3: low=1, high=1. mid=1. arr[1]=10 < 15 → low=2.

Step 4: low=2 > high=1. Loop ends. Return -1. (Not found)

Recursive Implementation

```
int binarySearchRec(int arr[], int low, int high, int key) {  
    if (low > high) return -1;           // Line 1  
    int mid = low + (high - low) / 2;    // Line 2  
    if (arr[mid] == key) return mid;     // Line 3  
    else if (arr[mid] < key) return binarySearchRec(arr, mid+1, high, key); // Line 4  
    else return binarySearchRec(arr, low, mid-1, key); // Line 5  
}
```

Complexity – Binary Search

Case	Time	Reasoning
Best	$O(1)$	Target at first mid
Average/Worst	$O(\log n)$	Halving gives $\log_2 n$ steps
Space (iterative)	$O(1)$	No extra memory
Space (recursive)	$O(\log n)$	Call stack depth

 Exam Tip: Binary search requires a SORTED array. State this precondition explicitly. The exam may ask for the number of comparisons for a specific search — trace through the dry run and count iterations.

5-Mark Answer Blueprint — Binary Search

- **Precondition:** array must be sorted
 - **Concept:** repeatedly halve the search space using mid comparison
 - **Code:** 10-15 lines, iterative version preferred in exams
 - **Dry run:** trace low, high, mid at each step on 6 values
 - **Complexity:** $O(\log n)$ time, $O(1)$ space (iterative)
-

3.3 Bubble Sort

Priority: Tier 1

Concept

Bubble sort repeatedly compares adjacent elements and swaps them if they're in the wrong order. The largest unsorted element "bubbles up" to its correct position at the end of each pass.

Algorithm

- For pass i from 0 to $n-2$:
 - For j from 0 to $n-2-i$:
 - If $\text{arr}[j] > \text{arr}[j+1]$: swap $\text{arr}[j]$ and $\text{arr}[j+1]$
- After $n-1$ passes, array is sorted.

Optimization: If no swaps occur in a pass, array is already sorted — break early.

C Implementation

```
void bubbleSort(int arr[], int n) {  
    for (int i = 0; i < n - 1; i++) {           // Line 1  
        int swapped = 0;                       // Line 2  
        for (int j = 0; j < n - 1 - i; j++) {   // Line 3  
            if (arr[j] > arr[j + 1]) {         // Line 4  
                int temp = arr[j];            // Line 5  
                arr[j] = arr[j + 1];          // Line 6  
                arr[j + 1] = temp;             // Line 7  
                swapped = 1;                   // Line 8  
            }  
        }  
        if (!swapped) break;                   // Line 9  
    }  
}
```

Line-by-line: 1. $n-1$ passes needed in worst case 2. Track if any swap happened this pass 3. Inner loop runs $n-1-i$ times (already-sorted tail grows by 1 each pass) 4. Adjacent comparison 5-7. Classic three-variable swap 8. Mark that a swap occurred 9. No swaps = array already sorted; exit early

Dry Run — Sort [64, 34, 25, 12, 22, 11]:

$n=6$. Array: [64, 34, 25, 12, 22, 11]

Pass 1 ($i=0$): compare $arr[j]$ and $arr[j+1]$ for $j=0..4$

$j=0$: $64 > 34$? YES. Swap. [34, 64, 25, 12, 22, 11]

$j=1$: $64 > 25$? YES. Swap. [34, 25, 64, 12, 22, 11]

$j=2$: $64 > 12$? YES. Swap. [34, 25, 12, 64, 22, 11]

j=3: 64>22? YES. Swap. [34, 25, 12, 22, 64, 11]

j=4: 64>11? YES. Swap. [34, 25, 12, 22, 11, 64]

After Pass 1: [34, 25, 12, 22, 11, 64] ← 64 in final position

Pass 2 (i=1): j=0..3

j=0: 34>25? YES. Swap. [25, 34, 12, 22, 11, 64]

j=1: 34>12? YES. Swap. [25, 12, 34, 22, 11, 64]

j=2: 34>22? YES. Swap. [25, 12, 22, 34, 11, 64]

j=3: 34>11? YES. Swap. [25, 12, 22, 11, 34, 64]

After Pass 2: [25, 12, 22, 11, 34, 64]

Pass 3 (i=2): j=0..2

j=0: 25>12? YES. [12, 25, 22, 11, 34, 64]

j=1: 25>22? YES. [12, 22, 25, 11, 34, 64]

j=2: 25>11? YES. [12, 22, 11, 25, 34, 64]

After Pass 3: [12, 22, 11, 25, 34, 64]

Pass 4 (i=3): j=0..1

j=0: 12>22? NO.

j=1: 22>11? YES. [12, 11, 22, 25, 34, 64]

After Pass 4: [12, 11, 22, 25, 34, 64]

Pass 5 (i=4): j=0

j=0: 12>11? YES. [11, 12, 22, 25, 34, 64]

After Pass 5: [11, 12, 22, 25, 34, 64] ← Sorted!

Complexity – Bubble Sort

Case	Comparisons	Time	Reason
Best (sorted input)	$n-1$	$O(n)$	1 pass, no swaps, break
Worst (reverse sorted)	$n(n-1)/2$	$O(n^2)$	All pairs compared and swapped
Average	$\sim n^2/4$	$O(n^2)$	
Space	—	$O(1)$	In-place
Stable?	Yes		Equal elements not swapped

📌 Exam Tip: Show the array state AFTER EACH COMPLETE PASS, not after each swap. The exam checks whether you know that the largest element reaches its final position after each pass. Count total passes and total comparisons for full marks.

3.4 Selection Sort

Priority: Tier 1

Concept

Selection sort finds the minimum element in the unsorted portion and places it at the beginning of that portion. After k passes, the first k elements are in their correct sorted positions.

Algorithm

- For i from 0 to n-2:
 - Find the index minIdx of the minimum element in arr[i..n-1]
 - Swap arr[i] with arr[minIdx]

C Implementation

```
void selectionSort(int arr[], int n) {  
    for (int i = 0; i < n - 1; i++) {           // Line 1  
        int minIdx = i;                        // Line 2  
        for (int j = i + 1; j < n; j++) {       // Line 3  
            if (arr[j] < arr[minIdx])           // Line 4  
                minIdx = j;                    // Line 5  
        }  
        if (minIdx != i) {                     // Line 6  
            int temp = arr[minIdx];            // Line 7  
            arr[minIdx] = arr[i];              // Line 8  
            arr[i] = temp;                     // Line 9  
        }  
    }  
}
```

Line-by-line: 1. $n-1$ passes 2. Assume current position has minimum 3. Scan rest of array for smaller element 4. Found smaller? 5. Update minimum index 6. Only swap if minimum is not already in position 7-9. Swap

Dry Run — Sort [64, 25, 12, 22, 11]:

$n=5$. Array: [64, 25, 12, 22, 11]

Pass 1 ($i=0$): Find min in [64,25,12,22,11]. $\text{minIdx}=4$ (value 11).

Swap $\text{arr}[0]$ and $\text{arr}[4]$: [11, 25, 12, 22, 64]

After Pass 1: [11, 25, 12, 22, 64] $\leftarrow$ 11 in final position

Pass 2 ($i=1$): Find min in [25,12,22,64]. $\text{minIdx}=2$ (value 12).

Swap $\text{arr}[1]$ and $\text{arr}[2]$: [11, 12, 25, 22, 64]

After Pass 2: [11, 12, 25, 22, 64]

Pass 3 ($i=2$): Find min in [25,22,64]. $\text{minIdx}=3$ (value 22).

Swap $\text{arr}[2]$ and $\text{arr}[3]$: [11, 12, 22, 25, 64]

After Pass 3: [11, 12, 22, 25, 64]

Pass 4 ($i=3$): Find min in [25,64]. $\text{minIdx}=3$ (value 25). Already in place.

No swap. After Pass 4: [11, 12, 22, 25, 64]

Final: [11, 12, 22, 25, 64] ✓

Complexity – Selection Sort

Case	Time	Space	Stable?
All cases	$O(n^2)$	$O(1)$	Not stable (equal elements may swap relative order)

Note: Selection sort makes at most $n-1$ swaps regardless of input, making it preferred when swap cost is high.

3.5 Insertion Sort

Priority: Tier 1

Concept

Insertion sort builds the sorted array one element at a time. For each element, it finds the correct position in the already-sorted left portion and inserts it there (shifting larger elements right).

Think of sorting a hand of playing cards: pick each card and insert it into the correct position among the already-sorted cards.

Algorithm

- For i from 1 to $n-1$:
 - $key = arr[i]$
 - $j = i - 1$
 - While $j \geq 0$ AND $arr[j] > key$: $arr[j+1] = arr[j]$; $j--$

- arr[j+1] = key

C Implementation

```
void insertionSort(int arr[], int n) {  
    for (int i = 1; i < n; i++) {           // Line 1  
        int key = arr[i];                  // Line 2  
        int j = i - 1;                     // Line 3  
        while (j >= 0 && arr[j] > key) {    // Line 4  
            arr[j + 1] = arr[j];           // Line 5  
            j--;                           // Line 6  
        }  
        arr[j + 1] = key;                  // Line 7  
    }  
}
```

Line-by-line: 1. Start from second element (first element trivially sorted) 2. Save the element to insert 3. Start comparing from just left of current position 4. Shift elements right while they're larger than key 5. Move arr[j] one position right 6. Move j left 7. Insert key at correct position

Dry Run — Sort [12, 11, 13, 5, 6]:

Array: [12, 11, 13, 5, 6]

i=1: key=11. j=0. arr[0]=12 > 11. arr[1]=12. j=-1. Place: arr[0]=11.

After: [11, 12, 13, 5, 6]

i=2: key=13. j=1. arr[1]=12 < 13. Stop. Place: arr[2]=13.

After: [11, 12, 13, 5, 6] (no change)

i=3: key=5. j=2. arr[2]=13>5 → arr[3]=13. j=1.

arr[1]=12>5 → arr[2]=12. j=0.

arr[0]=11>5 → arr[1]=11. j=-1.

Place: arr[0]=5.

After: [5, 11, 12, 13, 6]

i=4: key=6. j=3. arr[3]=13>6 → arr[4]=13. j=2.

arr[2]=12>6 → arr[3]=12. j=1.

arr[1]=11>6 → arr[2]=11. j=0.

arr[0]=5 < 6. Stop.

Place: arr[1]=6.

After: [5, 6, 11, 12, 13] ✓

Complexity – Insertion Sort

Case	Time	Condition
Best	$O(n)$	Already sorted, inner while never runs
Worst	$O(n^2)$	Reverse sorted, maximum shifts
Average	$O(n^2)$	Random input
Space	$O(1)$	In-place

Case	Time	Condition
Stable	Yes	Equal elements preserve relative order

📌 Exam Tip: Insertion sort is $O(n)$ on nearly-sorted data. Examiners often ask which sort is best for a nearly-sorted array – the answer is Insertion Sort. Also, state the invariant after each pass: “After pass i , the first $i+1$ elements are sorted among themselves.”

Sorting Algorithm Comparison Table

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes

3.6 Hashing

Priority: Tier 1 – Maximum Depth

Introduction

Hashing is a technique that maps a key to an index in an array (called a hash table) using a hash function. The goal is $O(1)$ average-case insertion, deletion, and lookup.

Hash Function $h(k)$ maps key $k \rightarrow$ index in table T

Key: "Alice" $\text{Hash}(\text{"Alice"}) = 3 \rightarrow T[3] = \text{"Alice's data"}$

Key: "Bob" $\text{Hash}(\text{"Bob"}) = 7 \rightarrow T[7] = \text{"Bob's data"}$

Key: "Charlie" $\text{Hash}(\text{"Charlie"}) = 3 \rightarrow \text{COLLISION with Alice!}$

Hash Table Structure

Hash Table T (size $m=7$):

Index: [0] [1] [2] [3] [4] [5] [6]

Data: [] [31] [] [Alice] [] [Bob] []

Key terms: - Hash function $h(k)$: maps key k to index $h(k)$ in range $[0, m-1]$ - Collision: when $h(k_1) == h(k_2)$ for $k_1 \neq k_2$ - Load factor $\alpha = n/m$: where n = number of stored elements, m = table size - Ideal load factor: $\alpha < 0.7$ for good performance

Hash Functions

1. Division Method (most common in exams):

$$h(k) = k \bmod m$$

Choose m as a prime number not close to a power of 2.

Example: $m=7$, keys = {23, 45, 12, 34, 67, 18}

$$h(23) = 23 \bmod 7 = 2$$

$$h(45) = 45 \bmod 7 = 3$$

$$h(12) = 12 \bmod 7 = 5$$

$$h(34) = 34 \bmod 7 = 6$$

$$h(67) = 67 \bmod 7 = 4$$

$$h(18) = 18 \bmod 7 = 4 \leftarrow \text{COLLISION with 67!}$$

2. Multiplication Method:

$$h(k) = \text{floor}(m * (k * A \bmod 1)) \text{ where } 0 < A < 1, \text{ Knuth suggests } A \approx 0.618$$

Example: $m=8, k=23, A=0.618$: $k * A = 23 * 0.618 = 14.214$ Fractional part: 0.214 $m * 0.214 = 1.712$ $\text{floor}(1.712) = 1$ $h(23) = 1$

3. Mid-Square Method:

Square the key, extract middle r digits (where $m = 10^r$).

Example: $k=3121, m=100$ (extract middle 2 digits): $k^2 = 3121^2 = 9740641$ Middle 2 digits = 40 (digits 3-4 from left) $h(3121) = 40$

4. Folding Method:

Divide key into equal-sized groups, add them together, apply mod m .

Example: $k=987654321$, group size=3, $m=1000$: Groups: $987 + 654 + 321 = 1962$ $h(k) = 1962 \bmod 1000 = 962$

Collision Resolution — Open Addressing

In open addressing, all elements are stored in the hash table itself. On collision, probe for the next available slot.

Linear Probing

Formula: $h(k, i) = (h(k) + i) \bmod m$, for $i = 0, 1, 2, \dots$

If slot $(h(k) + i) \bmod m$ is occupied, try $(h(k) + i + 1) \bmod m$.

C Implementation:

```
#define M 7

int hashTable[M];
int initialized = 0;

void initTable() {
    for (int i = 0; i < M; i++) hashTable[i] = -1; // -1 = empty
    initialized = 1;
}

int hashFunc(int key) { return key % M; }

void linearProbeInsert(int key) {
    int h = hashFunc(key);           // Line 1
    int i = 0;                       // Line 2
    while (i < M) {                  // Line 3
        int pos = (h + i) % M;       // Line 4
        if (hashTable[pos] == -1) { // Line 5
            hashTable[pos] = key;    // Line 6
            printf("Inserted %d at index %d\n", key, pos); // Line 7
            return;
        }
        i++;                         // Line 8
    }
    printf("Hash table full\n");    // Line 9
}
```

```

int linearProbeSearch(int key) {
    int h = hashFunc(key);
    for (int i = 0; i < M; i++) {
        int pos = (h + i) % M;
        if (hashTable[pos] == key) return pos;    // found
        if (hashTable[pos] == -1) return -1;    // empty slot – not in table
    }
    return -1;
}

```

Line-by-line (insert): 1. Compute initial hash index 2. Probe counter starts at 0 3. Try at most M probes 4. Compute probed position with modular wrap 5. Empty slot found 6. Store key 7. Report 8. Advance probe counter 9. Table is full

Dry Run — Linear Probing (m=7), Insert: 23, 45, 30, 36, 16, 22:

Hash table T: [-1, -1, -1, -1, -1, -1, -1] (index 0-6)

Insert 23: $h(23) = 23\%7 = 2$. $T[2]=-1 \rightarrow T[2]=23$.

T: [-1, -1, 23, -1, -1, -1, -1]

Insert 45: $h(45) = 45\%7 = 3$. $T[3]=-1 \rightarrow T[3]=45$.

T: [-1, -1, 23, 45, -1, -1, -1]

Insert 30: $h(30) = 30\%7 = 2$. $T[2]=23 \neq -1$. COLLISION!

Probe i=1: $pos = (2+1)\%7 = 3$. $T[3]=45 \neq -1$. COLLISION!

Probe i=2: $pos = (2+2)\%7 = 4$. $T[4]=-1 \rightarrow T[4]=30$.

T: [-1, -1, 23, 45, 30, -1, -1]

Insert 36: $h(36) = 36\%7 = 1$. $T[1]=-1 \rightarrow T[1]=36$.

T: [-1, 36, 23, 45, 30, -1, -1]

Insert 16: $h(16) = 16\%7 = 2$. $T[2]=23$. Collision. $i=1$: $T[3]=45$. $i=2$: $T[4]=30$. $i=3$: $T[5]=-1 \rightarrow T[5]=16$.

T: [-1, 36, 23, 45, 30, 16, -1]

Insert 22: $h(22) = 22\%7 = 1$. $T[1]=36$. Collision. $i=1$: $T[2]=23$. $i=2$: $T[3]=45$. $i=3$: $T[4]=30$. $i=4$: $T[5]=16$. $i=5$: $T[6]=-1 \rightarrow T[6]=22$.

T: [-1, 36, 23, 45, 30, 16, 22]

Final Hash Table:

Index: [0] [1] [2] [3] [4] [5] [6]

Data: [-1] [36] [23] [45] [30] [16] [22]

Search 16: $h(16)=2$. $T[2]=23 \neq 16$. $T[3]=45 \neq 16$. $T[4]=30 \neq 16$. $T[5]=16$ ✓ Found at index 5.

 Exam Tip: In linear probing dry runs, show every probe step with the index computed and whether it was a hit or collision. Count total probes per insertion. The examiner checks whether you correctly apply the $(h+i)\%m$ formula with wrap-around.

Quadratic Probing

Formula: $h(k, i) = (h(k) + i^2) \bmod m$, for $i = 0, 1, 2, \dots$

Reduces primary clustering (consecutive filled slots) that occurs with linear probing.

Dry Run ($m=7$): Insert 18, then 25, both hash to $h=4$:

$h(18) = 18\%7 = 4$. $T[4] = 18$.

$h(25) = 25\%7 = 4$. Collision.

$i=1$: $(4+1)\%7 = 5$. $T[5]=\text{empty} \rightarrow T[5]=25$.

With linear probing this would also go to 5. But for a longer cluster:

If $T = [_, _, _, _, 18, 25, _]$:

Insert key k with $h(k)=4$:

Linear: probes 4,5,6,0,1,2 (sequential – hits cluster)

Quadratic: probes 4, $(4+1)=5$, $(4+4)=8\%7=1$ (jumps over cluster)

C snippet:

```
int pos = (h + i*i) % M; // in the while loop
```

Double Hashing

Formula: $h(k, i) = (h_1(k) + i * h_2(k)) \bmod m$

Uses a second hash function $h_2(k)$ to determine step size. Virtually eliminates clustering.

Common choice: $h_1(k) = k \bmod m$, $h_2(k) = 1 + (k \bmod (m-1))$

Dry Run ($m=7$): Insert 18 and 25 (both hash to 4):

$h_1(18) = 18\%7 = 4$. $h_2(18) = 1 + (18\%6) = 1+0 = 1$.

$h_1(25) = 25\%7 = 4$. $h_2(25) = 1 + (25\%6) = 1+1 = 2$.

Insert 18: $i=0$, $\text{pos}=4$. $T[4]=18$.

Insert 25: $i=0$, $\text{pos}=4$. Collision.

$i=1$: $\text{pos} = (4 + 1*2)\%7 = 6$. $T[6]=\text{empty} \rightarrow T[6]=25$.

C snippet:

```
int h1 = key % M;  
int h2 = 1 + (key % (M - 1));  
int pos = (h1 + i * h2) % M; // in the probe loop
```

Collision Resolution — Separate Chaining

Concept: Instead of finding another slot in the table, each table slot holds a linked list of all keys that hash to that index.

Hash Table (m=7) with separate chaining:

Index 0: NULL

Index 1: [36] -> NULL

Index 2: [23] -> [30] -> NULL (30 also hashed to 2)

Index 3: [45] -> NULL

Index 4: [67] -> [18] -> NULL (18 also hashed to 4)

Index 5: NULL

Index 6: NULL

C Implementation:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define M 7
```

```
struct ChainNode {
```

```
    int data;
```

```
    struct ChainNode *next;
```

```
};
```

```
struct ChainNode* table[M];
```

```
void initChainTable() {  
    for (int i = 0; i < M; i++) table[i] = NULL;  
}
```

```
int hashFunc(int key) { return key % M; }
```

```
void chainInsert(int key) {  
    int h = hashFunc(key);                // Line 1  
    struct ChainNode* newNode = (struct ChainNode*)malloc(sizeof(struct  
ChainNode)); // Line 2  
    newNode->data = key;                   // Line 3  
    newNode->next = table[h];              // Line 4  
    table[h] = newNode;                   // Line 5  
}
```

```
int chainSearch(int key) {  
    int h = hashFunc(key);  
    struct ChainNode* curr = table[h];  
    while (curr != NULL) {  
        if (curr->data == key) return h;  
        curr = curr->next;  
    }  
    return -1;  
}
```

```

void displayChainTable() {
    for (int i = 0; i < M; i++) {
        printf("T[%d]: ", i);
        struct ChainNode* curr = table[i];
        while (curr != NULL) { printf("%d -> ", curr->data); curr = curr->next; }
        printf("NULL\n");
    }
}

```

Line-by-line (insert): 1. Compute hash index 2. Allocate new node 3. Store key 4. New node points to current head of chain (insert at beginning for $O(1)$) 5. Table entry now points to new node

Dry Run — Separate Chaining ($m=7$), Insert: 23, 45, 30, 36, 16, 22:

$h(23)=2$: $T[2] = [23] \rightarrow \text{NULL}$

$h(45)=3$: $T[3] = [45] \rightarrow \text{NULL}$

$h(30)=2$: $T[2] = [30] \rightarrow [23] \rightarrow \text{NULL}$ (30 inserted at front)

$h(36)=1$: $T[1] = [36] \rightarrow \text{NULL}$

$h(16)=2$: $T[2] = [16] \rightarrow [30] \rightarrow [23] \rightarrow \text{NULL}$

$h(22)=1$: $T[1] = [22] \rightarrow [36] \rightarrow \text{NULL}$

Final Table:

$T[0]$: NULL

$T[1]$: 22 -> 36 -> NULL

$T[2]$: 16 -> 30 -> 23 -> NULL

$T[3]$: 45 -> NULL

$T[4]$: NULL

T[5]: NULL

T[6]: NULL

 Exam Tip: Separate chaining vs. open addressing — know the key differences. Chaining handles any load factor, never suffers primary/secondary clustering, but uses extra memory for pointers. Open addressing keeps all data in-table but degrades severely at high load factors. State α and which method is preferred for each case.

Comparison — Collision Resolution Methods

Feature	Linear Probing	Quadratic Probing	Double Hashing	Separate Chaining
Probe formula	$(h+i)\%m$	$(h+i^2)\%m$	$(h+i*h_2)\%m$	N/A (linked list)
Primary clustering	Severe	Reduced	None	None
Secondary clustering	None	Present	None	None
Load factor limit	$\alpha < 0.7$	$\alpha < 0.7$	$\alpha < 0.7$	Any α

Feature	Linear Probing	Quadratic Probing	Double Hashing	Separate Chaining
Memory	In-table	In-table	In-table	Extra (pointers)
Cache performance	Best	Good	Good	Poor
Implementation	Simple	Moderate	Moderate	Moderate

Time Complexity – Hashing

Operation	Average Case	Worst Case
Insert	$O(1)$	$O(n)$ (all keys collide)
Search	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$

5-Mark Answer Blueprint – Hashing with Linear Probing

- **Hash function:** $h(k) = k \bmod m$, with example
- **Collision definition:** two keys mapping to same index

- **Linear probing formula:** $h(k,i) = (h(k)+i) \bmod m$
 - **Dry run table:** show all insertions with collision handling, produce final table
 - **Complexity:** $O(1)$ average, $O(n)$ worst for open addressing
 - **Load factor note:** performance degrades when $\alpha > 0.7$
-

Summary / TL;DR – CIE 2 Exam Quick Reference

Module 2 – Linked Lists: - CSLL: last->next = head; traversal uses do-while; all ops $O(n)$; NULL never appears - CDLL: last->next = head AND head->prev = last; insert-begin/end both $O(1)$ using head->prev - Polynomial addition: always 3 cases – $\text{exp1} > \text{exp2}$, $\text{exp1} < \text{exp2}$, $\text{exp1} == \text{exp2}$ (add coefficients) - Polynomial multiplication: $O(m*n)$ term pairs; merge same-exponent terms in result

Module 3 – Stacks and Queues: - Infix→Postfix: operand→output; (→push;)→pop until (; operator→pop higher/equal prec then push - Postfix eval: operand→push; operator→pop b then a, compute a OP b, push result - Circular Queue: $\text{rear} = (\text{rear} + 1) \% \text{MAX}$; $\text{front} = (\text{front} + 1) \% \text{MAX}$; full: $(\text{rear} + 1) \% \text{MAX} == \text{front}$

Module 4 – Sort/Search/Hash: - Binary Search: $O(\log n)$, needs sorted array, $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$ - Bubble/Selection/Insertion: all $O(n^2)$ worst, $O(1)$ space; Bubble+Insertion stable - Insertion Sort: $O(n)$ best (nearly sorted); Selection: min swaps (at most $n-1$) - Hashing: $h(k) = k \% m$; Linear probing: $(h+i) \% m$; Separate chaining: linked lists per slot